

claude-windows / d1586dfe-adae-4d40-a85d-ed0bb23738b8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\d1586dfe-adae-4d40-a85d-ed0bb23738b8\subagents\agent-afa9c802e3657ad51.jsonl`  
- SHA-256: `47700b54d392aa01001395199c01739d2a1084a11bab9e7226abaa5b31d95bed`  
- Source modified: `2026-06-10T18:16:31+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `subagents`  
- session_id: `d1586dfe-adae-4d40-a85d-ed0bb23738b8`
```

Transcript

1. user (2026-06-10T17:59:40.675Z)

You are working in C:\Users\avidu\Projects\gmail-attachments-to-gdrive ? a Google Apps Script project in TypeScript (esbuild bundle, Jest tests with an 85%/75% coverage gate, pure-logic/GAS-shell separation: all logic is pure and unit-tested; GAS-facing code is a thin shell). Current branch should be master and clean ? verify with git status first; create feature branches off master. Node: run `npm ci` if node_modules is missing, then `npm test` to see the baseline.

MISSION: fix three verified data-loss/correctness findings from a deep review (2026-06). Read the relevant sources COMPLETELY before changing anything: src/sync.ts, src/historySync.ts, src/gmailHistory.ts, src/drive.ts, src/state.ts, src/processed.ts, src/config.ts, src/query.ts, appsscript.json, plus the matching tests in tests/. Honor the repo's design principles (ROADMAP.md "Platform constraints", docs/ARCHITECTURE.md "Principles"): quota discipline, "mark a message done only on clean completion", pure functions + thin shells, heavy unit testing of the pure parts.

FINDINGS TO FIX (verified 3/3 by adversarial review):

****F1 (high) ? backfill advances lastSynced past windows with attachment-level errors****
(src/sync.ts around lines 213-218): when updateDrive reports per-message/attachment errors (transient Drive failures, daily-cap truncation), the date-window cursor still advances past the window, so failed messages are NEVER retried. Fix so a window only advances when it completed cleanly (no attachment-level errors and not truncated by the daily cap); a dirty window must be re-scanned next run (the processed-ids FIFO + the Drive getFilesByName probe already make re-scans cheap and duplicate-safe ? lean on that, do not invent a new retry queue). Mind the 6-minute budget: a permanently-failing message must not wedge the cursor forever ? if the SAME window stays dirty across many runs that is acceptable to ship as-is ONLY if you surface it via the existing logger/stats (e.g. a counter of consecutive dirty passes); choose the simplest mechanism consistent with the codebase.

****F2 (high+medium) ? incremental history sync loses/wedges****: (a) src/sync.ts ~175-189: the historyId cursor advances even when messages were truncated by the daily cap or failed transiently ? silent data loss; only advance past what was fully processed (the Gmail history API gives per-record historyId ? study src/gmailHistory.ts/historySync.ts to find the right anchor; if per-record anchoring is infeasible, do NOT advance the cursor on a dirty run). (b) src/gmailHistory.ts ~70-77: GmailApp.getMessageById THROWS for deleted/inaccessible messages (it does not return null) ? wrap it so one deleted message skips cleanly instead of wedging the whole incremental loop until cursor expiry. (c) the incremental path applies no spam/trash filtering while the backfill query excludes them ? add parity (check message labels/thread properties available via the advanced service metadata already fetched; keep it cheap, no extra quota calls if the data is already in hand).

****F3 (medium/low) ? operational defaults****: (a) src/drive.ts getOrCreateFolderInFolder called with DriveApp as the parent iterates ALL folders in the user's Drive (DriveApp.getFolders() is global, not root children) ? use DriveApp.getRootFolder() as the actual parent so the search is scoped to root children. (b) appsscript.json timeZone is "America/Los_Angeles" ? the owner is in Bangalore; change to "Asia/Kolkata" (day boundaries / cap resets / summary rollover follow it).

(c) src/config.ts SUMMARY_EMAIL_RECIPIENT is a hardcoded global ? in a multi-user deployment every user's summary is mailed (as them) to one address; make the recipient default to the executing user (Session.getActiveUser().getEmail() at the shell layer, threaded as a parameter into the pure parts) with the constant only as an explicit override if it is non-empty. Follow the existing pure/shell pattern for testability.

SHIP AS TWO PRs:

- Branch `fix/sync-data-loss` (F1+F2): commit, push -u, `gh pr create` with a body explaining each fix + the invariant it restores.

- Branch `fix/operational-defaults` (F3, branched off master, independent): same.

Both commit messages end with the trailer "Co-Authored-By: Claude Fable 5

<noreply@anthropic.com>"; both PR bodies end with "? Generated with [Claude Code](https://claude.com/claude-code)".

****Do NOT merge either PR**** ? merging to master auto-deploys to the user's live Apps Script; the user reviews these.

QUALITY BAR: every behavior change gets a Jest test in the pure layer (extend the existing test files; follow their style); `npm test` green with the coverage thresholds; `npm run build` (or the esbuild script in package.json) must succeed. Reason about Apps Script runtime semantics, not generic TS (e.g. GmailApp/Drive advanced-service behaviors). Keep diffs minimal and idiomatic to the codebase. If something in a finding turns out to be wrong once you read the code (the review could err), say so in the PR body and fix what is actually true instead.

Your final message: the two PR URLs, what changed per fix, test counts before/after, and any finding you adjusted. Raw facts.

2. user (2026-06-10T17:59:47.603Z)

On branch master

Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

* master

remotes/origin/HEAD -> origin/master

remotes/origin/master

a764030 Codify the auditability invariant (Option A) + enforce no egress (#14)

90c85ba Merge pull request #13 from avidullu/docs/vision-and-architecture

4975254 docs: capture extraction-pivot vision and architecture

172b3b3 Validate setup input + traceable attachment-copy errors (#12)

964b58c Portable attachment names + robust content-type matching (#11)

3. user (2026-06-10T17:59:47.674Z)

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\description

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\applypatch-msg.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\commit-msg.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\fsmonitor-watchman.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\post-update.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-applypatch.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-commit.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-merge-commit.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-push.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-rebase.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-receive.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\prepare-commit-msg.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\push-to-checkout.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\sendemail-validate.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\update.sample

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\info\exclude

C:\Users\avidu\Projects\gmail-attachments-to-

gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.idx

C:\Users\avidu\Projects\gmail-attachments-to-

gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.pack

C:\Users\avidu\Projects\gmail-attachments-to-

gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.rev

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\remotes\origin\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\packed-refs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\remotes\origin\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.clasp.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.claspignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\config
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.github\workflows\ci.yml
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.gitignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\appsscript.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\ARCHITECTURE.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\VISION.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\esbuild.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\eslint.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\LICENSE
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\PRIVACY.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\README.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\ROADMAP.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\html\index.html
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\jest.config.js
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package-lock.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\config.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\contentTypes.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\entry.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\logger.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\naming.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\processed.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\query.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\quota.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\stats.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\types\gmailAdvanced.d.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\ui.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\auditability.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\contentTypes.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\naming.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\processed.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\quota.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\setup.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\stats.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tsconfig.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\index

4. user (2026-06-10T17:59:52.564Z)

```
1 import {  
2   ENABLE_INCREMENTAL_SYNC,  
3   MAX_FILES_ADD_PER_DAY,
```

```

4      MAX_RUNTIME_MS,
5      MAX_SYNC_TRIGGERS,
6      ONE_DAY,
7      ONE_MINUTE,
8      SIX_HOURS,
9      THIRTY_DAYS,
10   } from "./config";
11   import { updateDrive, type UpdateResult } from "./drive";
12   import {
13     getCurrentHistoryId,
14     listAddedSince,
15     threadsForMessageIds,
16   } from "./gmailHistory";
17   import { logDebug, logError } from "./logger";
18   import { maybeSendDailySummary } from "./notify";
19   import { addProcessedId } from "./processed";
20   import { buildSearchQuery } from "./query";
21   import { hasTime, isSameCalendarDay } from "./quota";
22   import { dayKey, recordRun } from "./stats";
23   import * as state from "./state";
24
25   const SYNC_HANDLER = "syncUserDrives";
26
27   /** Schedules a one-shot run of syncUserDrives `afterMs` from now. */
28   export function newSyncTrigger(afterMs: number): void {
29     ScriptApp.newTrigger(SYNC_HANDLER).timeBased().after(afterMs).create();
30   }
31
32   /** All currently-installed triggers for the sync handler. */
33   function getSyncTriggers(): GoogleAppsScript.Script.Trigger[] {
34     return ScriptApp.getProjectTriggers().filter(
35       (t) => t.getHandlerFunction() === SYNC_HANDLER,
36     );
37   }
38
39   /** Removes every project trigger (used when the user re-runs setup). */
40   export function deleteAllTriggers(): void {
41     for (const trigger of ScriptApp.getProjectTriggers()) {
42       ScriptApp.deleteTrigger(trigger);
43     }
44   }
45
46   function deleteTriggers(triggers: GoogleAppsScript.Script.Trigger[]): void {
47     for (const trigger of triggers) {
48       ScriptApp.deleteTrigger(trigger);
49     }
50   }
51
52   /** Defense-in-depth: never let sync triggers exceed the cap (limit is 20). */
53   function pruneSyncTriggers(max: number): void {
54     const triggers = getSyncTriggers();
55     for (let i = max; i < triggers.length; i++) {
56       ScriptApp.deleteTrigger(triggers[i]);
57     }
58   }
59
60   /**
61    * The background worker. A per-user lock ensures only one run executes at a time
62    * for THIS user, so concurrent triggers cannot stack file copies or trigger
63    * generations. It is deliberately a user lock, not a script lock: the web app
64    * runs as the accessing user (executeAs USER_ACCESSING), so a script-wide lock
65    * would serialise every user's sync into one global queue.
66    */
67   export function syncUserDrives(): void {
68     const lock = LockService.getUserLock();

```

```

69     if (!lock.tryLock(0)) {
70         logDebug("Another sync run is in progress; skipping.");
71         return;
72     }
73     try {
74         runSyncOnce();
75     } finally {
76         // Bound the trigger count regardless of which path we took, then release.
77         pruneSyncTriggers(MAX_SYNC_TRIGGERS);
78         lock.releaseLock();
79     }
80 }
81
82 /**
83  * A single sync pass:
84  *   1. reschedules a fan-out of backup triggers (self-healing if one is lost),
85  *   2. syncs forward in 30-day windows until it runs out of time, hits the
86  *       daily file cap, or fully catches up,
87  *   3. persists progress and schedules the next run.
88  */
89 function runSyncOnce(): void {
90     const today = new Date();
91
92     // On the first run of a new calendar day, summarise the previous day.
93     maybeSendDailySummary(today);
94
95     // Recover state if the debug flag was cleared (e.g. properties were wiped).
96     const debug = state.getDebug();
97     if (debug == null || debug === "0") {
98         state.setDebug("1");
99         state.setLastSynced(new Date(today.getTime() - 365 * ONE_DAY).toJSON());
100     }
101
102     // Create the backup triggers BEFORE deleting the current ones, so a failure
103     // here can never leave us with no scheduled run.
104     const previousTriggers = getSyncTriggers();
105     try {
106         newSyncTrigger(60 * ONE_MINUTE);
107         newSyncTrigger(SIX_HOURS);
108         newSyncTrigger(2 * SIX_HOURS);
109         newSyncTrigger(4 * SIX_HOURS);
110         newSyncTrigger(8 * SIX_HOURS);
111         newSyncTrigger(28 * SIX_HOURS);
112     } catch (e) {
113         // Could not install new triggers; leave the old ones and try again later.
114         logError("Could not install triggers: " + e);
115         return;
116     }
117     deleteTriggers(previousTriggers);
118
119     const filesAddedToday = state.getFilesAddedToday();
120     let added = filesAddedToday.added;
121     const lastCountDate = new Date(Date.parse(filesAddedToday.date));
122
123     if (isSameCalendarDay(lastCountDate, today) && added >= MAX_FILES_ADD_PER_DAY) {
124         logDebug("Used up all quota for today. Can't add more files.");
125         newSyncTrigger(60 * ONE_MINUTE);
126         return;
127     }
128     if (!isSameCalendarDay(lastCountDate, today)) {
129         added = 0;
130     }
131
132     const structure = state.getFolderStructure();
133     let lastSynced = state.getLastSynced();

```

```

134
135 // Load the dedup set once; thread it through the run and persist at the end.
136 let processedIds = state.getProcessedIds();
137 const processedSet = new Set(processedIds);
138
139 // This run's contribution to the day's stats.
140 const addedAtStart = added;
141 let runErrors = 0;
142 let runLastError = "";
143
144 // Folds one updateDrive result into this run's accumulators.
145 const applyResult = (result: UpdateResult): void => {
146     added += result.addedFiles;
147     runErrors += result.errors;
148     if (result.lastError) {
149         runLastError = result.lastError;
150     }
151     for (const id of result.processedIds) {
152         if (!processedSet.has(id)) {
153             processedSet.add(id);
154             processedIds = addProcessedId(processedIds, id);
155         }
156     }
157 };
158
159 try {
160     // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
161     // gradually over the proven date-window backfill. A stored historyId only
162     // exists once a backfill has caught up while the flag was on.
163     const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
164     if (historyId) {
165         // Incremental: process only messages added since the stored historyId.
166         const changes = listAddedSince(historyId);
167         if (changes == null) {
168             // History expired: clear the cursor and re-scan a bounded recent window
169             // (the next catch-up re-captures a fresh historyId). Overlap is safe ?
170             // dedup by message id and the Drive filename guard prevent re-copying.
171             state.clearHistoryId();
172             lastSynced = new Date(today.getTime() - THIRTY_DAYS).toJSON();
173             logDebug("History expired; reverting to date-window sync.");
174         } else {
175             if (changes.messageIds.length > 0 && added < MAX_FILES_ADD_PER_DAY) {
176                 const threads = threadsForMessageIds(changes.messageIds);
177                 if (threads.length > 0) {
178                     applyResult(
179                         updateDrive(
180                             threads,
181                             structure,
182                             MAX_FILES_ADD_PER_DAY - added,
183                             processedSet,
184                         ),
185                     );
186                 }
187             }
188             // Advance the cursor so the next run only sees newer changes.
189             state.setHistoryId(changes.historyId);
190         }
191     } else {
192         // Backfill: walk forward in 30-day windows until caught up to today.
193         // Capture the mailbox historyId BEFORE processing so mail arriving
194         // mid-run is still picked up by the first incremental scan.
195         const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
196             ? getCurrentHistoryId()
197             : null;
198         let fullSynced = false;

```

```

199     while (
200         hasTime(new Date(), today, MAX_RUNTIME_MS) &&
201         added < MAX_FILES_ADD_PER_DAY &&
202         !fullSynced
203     ) {
204         const maxToAdd = MAX_FILES_ADD_PER_DAY - added;
205         const search = buildSearchQuery(lastSynced, new Date());
206         logDebug("Query: " + search.query + " files asking: " + maxToAdd);
207
208         const threads = GmailApp.search(search.query);
209         if (threads.length > 0) {
210             applyResult(updateDrive(threads, structure, maxToAdd, processedSet));
211         }
212
213         fullSynced = search.isToday;
214         if (added < MAX_FILES_ADD_PER_DAY) {
215             lastSynced = fullSynced
216                 ? today.toJSON()
217                 : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();
218         }
219     }
220     // Caught up: switch future runs to the cheaper incremental path.
221     if (candidateHistoryId && fullSynced && added < MAX_FILES_ADD_PER_DAY) {
222         state.setHistoryId(candidateHistoryId);
223         logDebug("Backfill complete; stored historyId " + candidateHistoryId);
224     }
225 }
226 } catch (e) {
227     logError("Some error in execution: " + e);
228     state.setLastError(String(e));
229     runErrors++;
230     runLastError = String(e);
231 }
232
233 // Fold this run into the day's accumulator.
234 const dayStats = recordRun(state.getRunStats(dayKey(today)), {
235     filesAdded: added - addedAtStart,
236     errors: runErrors,
237     lastError: runLastError,
238 });
239 state.setRunStats(dayStats);
240
241 state.setProcessedIds(processedIds);
242 state.setLastSynced(lastSynced);
243 state.setFilesAddedToday({ added, date: today.toJSON() });
244 newSyncTrigger(20 * ONE_MINUTE);
245 }
246

```

5. user (2026-06-10T17:59:53.034Z)

```

1  /*
2   * Pure helpers for the Gmail `users.history.list` incremental sync. These
3   * operate on plain data shapes (the JSON the Gmail advanced service returns)
4   * and have no GAS dependencies, so they are exercised directly in unit tests.
5   * The actual advanced-service calls live in the `gmailHistory` shell.
6   *
7   * See https://developers.google.com/workspace/gmail/api/guides/sync ? a stored
8   * `startHistoryId` returns only the changes since that point (~2 quota units vs
9   * ~5 for a list call). The id can expire, in which case the API responds 404
10  * and the caller must fall back to a full (date-window) sync.
11  */
12
13 /** A message stub as it appears inside a history record. */
14 export interface HistoryMessage {

```

```

15     id?: string;
16     threadId?: string;
17 }
18
19 /** One change record; we only care about messages added to the mailbox. */
20 export interface HistoryRecord {
21     messagesAdded?: { message?: HistoryMessage }[];
22 }
23
24 /** The shape of a `Gmail.Users.History.list` response we depend on. */
25 export interface HistoryListResponse {
26     history?: HistoryRecord[];
27     historyId?: string;
28     nextPageToken?: string;
29 }
30
31 /**
32  * Collects the unique ids of messages added across the given history pages, in
33  * first-seen order. Records without a `messagesAdded` entry (e.g. label-only
34  * changes) contribute nothing.
35  */
36 export function collectAddedMessageIds(
37     responses: HistoryListResponse[],
38 ): string[] {
39     const seen = new Set<string>();
40     const ids: string[] = [];
41     for (const response of responses) {
42         for (const record of response.history ?? []) {
43             for (const added of record.messagesAdded ?? []) {
44                 const id = added.message?.id;
45                 if (id && !seen.has(id)) {
46                     seen.add(id);
47                     ids.push(id);
48                 }
49             }
50         }
51     }
52     return ids;
53 }
54
55 /**
56  * The newest `historyId` to persist for the next run: the last page's id, or
57  * the supplied fallback (the start id) if no page reported one.
58  */
59 export function nextHistoryId(
60     responses: HistoryListResponse[],
61     fallback: string,
62 ): string {
63     for (let i = responses.length - 1; i >= 0; i--) {
64         const id = responses[i].historyId;
65         if (id) {
66             return id;
67         }
68     }
69     return fallback;
70 }
71
72 /**
73  * Whether an error thrown by `history.list` means the stored `startHistoryId`
74  * has expired. Gmail responds with HTTP 404 ("Requested entity was not found")
75  * once the id predates the mailbox's retained history; the caller must then do
76  * a full sync. We match defensively on both the status code and the message
77  * text since the advanced service surfaces errors as a stringified exception.
78  */
79 export function isHistoryExpired(error: unknown): boolean {

```

```

80     const e = error as { message?: string; details?: { code?: number } } | null;
81     if (e && e.details && e.details.code === 404) {
82         return true;
83     }
84     const text = String(e?.message ?? error ?? "").toLowerCase();
85     return text.includes("404") || text.includes("not found");
86 }
87

```

6. user (2026-06-10T17:59:53.476Z)

```

1     import {
2         collectAddedMessageIds,
3         isHistoryExpired,
4         nextHistoryId,
5         type HistoryListResponse,
6     } from "./historySync";
7     import { logDebug } from "./logger";
8
9     type GmailThread = GoogleAppsScript.Gmail.GmailThread;
10
11     /** A bounded number of history pages to walk in a single run. */
12     const MAX_HISTORY_PAGES = 20;
13
14     /** The mailbox's current historyId, used to seed incremental sync. */
15     export function getCurrentHistoryId(): string | null {
16         const profile = Gmail.Users.getProfile("me");
17         return profile.historyId ?? null;
18     }
19
20     /** What an incremental scan yields: new message ids and the id to store next. */
21     export interface IncrementalChanges {
22         messageIds: string[];
23         historyId: string;
24     }
25
26     /**
27      * Lists messages added since `startHistoryId` via the Gmail advanced service,
28      * paging through results. Returns null when the history has expired (the caller
29      * must fall back to a full sync). Only `messageAdded` events are requested.
30      */
31     export function listAddedSince(
32         startHistoryId: string,
33     ): IncrementalChanges | null {
34         const responses: HistoryListResponse[] = [];
35         let pageToken: string | undefined;
36         try {
37             for (let page = 0; page < MAX_HISTORY_PAGES; page++) {
38                 const response = Gmail.Users.History.list("me", {
39                     startHistoryId,
40                     historyTypes: ["messageAdded"],
41                     pageToken,
42                 }) as HistoryListResponse;
43                 responses.push(response);
44                 if (!response.nextPageToken) {
45                     break;
46                 }
47                 pageToken = response.nextPageToken;
48             }
49         } catch (e) {
50             if (isHistoryExpired(e)) {
51                 logDebug("History id expired; full sync required.");
52                 return null;
53             }
54             throw e;
55         }
56     }
57

```

```

55     }
56     return {
57         messageIds: collectAddedMessageIds(responses),
58         historyId: nextHistoryId(responses, startHistoryId),
59     };
60 }
61
62 /**
63  * Resolves the distinct Gmail threads containing the given message ids. Threads
64  * are de-duplicated by id so a multi-message thread is processed once. Messages
65  * that can no longer be fetched (deleted since the change was recorded) are
66  * skipped.
67  */
68 export function threadsForMessageIds(messageIds: string[]): GmailThread[] {
69     const threads = new Map<string, GmailThread>();
70     for (const id of messageIds) {
71         const message = GmailApp.getMessageById(id);
72         if (!message) {
73             continue;
74         }
75         const thread = message.getThread();
76         threads.set(thread.getId(), thread);
77     }
78     return Array.from(threads.values());
79 }
80

```

7. user (2026-06-10T17:59:58.065Z)

```

1     import { GMAIL_ATTACHMENT_FOLDER } from "./config";
2     import { isAllowedContentType } from "./contentTypes";
3     import { getAttachmentName, getFolderNameForStructure } from "./naming";
4     import { logDebug, logError } from "./logger";
5
6     type Folder = GoogleAppsScript.Drive.Folder;
7
8     /**
9      * Anything that can hold folders: both `DriveApp` (the root) and a `Folder`
10     * expose `getFolders()` / `createFolder()`, so we accept either structurally.
11     */
12     interface FolderContainer {
13         getFolders(): GoogleAppsScript.Drive.FolderIterator;
14         createFolder(name: string): Folder;
15     }
16
17     export interface UpdateResult {
18         /** Number of files actually written to Drive in this call. */
19         addedFiles: number;
20         hasError: boolean;
21         /** Count of attachment-level errors in this call. */
22         errors: number;
23         /** Most recent error message, if any. */
24         lastError: string;
25         /** Gmail message ids fully handled in this call (for cross-run dedup). */
26         processedIds: string[];
27     }
28
29     /** Returns the named child folder, creating it if it does not exist yet. */
30     export function getOrCreateFolderInFolder(
31         parent: FolderContainer,
32         name: string,
33     ): Folder {
34         const folders = parent.getFolders();
35         while (folders.hasNext()) {
36             const folder = folders.next();

```

```

37     if (folder.getName() === name) {
38         logDebug("Found existing folder " + name);
39         return folder;
40     }
41 }
42 logDebug("Creating new folder " + name);
43 return parent.createFolder(name);
44 }
45
46 /**
47  * Resolves the destination sub-folder for an attachment, creating it under
48  * `head` and caching it in `existingFolders`. Returns null when the chosen
49  * structure yields no folder (caller falls back to the head folder).
50  */
51 function resolveFolder(
52     head: Folder,
53     existingFolders: Map<string, Folder>,
54     structure: string,
55     contentType: string,
56     date: Date,
57 ): Folder | null {
58     const name = getFolderNameForStructure(structure, contentType, date);
59     if (name === "") {
60         return null;
61     }
62     const cached = existingFolders.get(name);
63     if (cached) {
64         return cached;
65     }
66     const created = head.createFolder(name);
67     existingFolders.set(name, created);
68     return created;
69 }
70
71 /**
72  * Copies every allowed, not-yet-present attachment from the given threads into
73  * the GmailAttachments folder, organised per the user's chosen structure.
74  * Stops once `maxToAdd` files have been written.
75  *
76  * `alreadyProcessed` is the set of Gmail message ids handled in previous runs;
77  * such messages are skipped wholesale, avoiding a Drive `getFilesByName` probe
78  * per attachment when an overlapping date window is re-scanned. Ids fully
79  * handled here are returned in `processedIds` for the caller to persist.
80  */
81 export function updateDrive(
82     threads: GoogleAppsScript.Gmail.GmailThread[],
83     structure: string,
84     maxToAdd: number,
85     alreadyProcessed: ReadonlySet<string> = new Set(),
86 ): UpdateResult {
87     const head = getOrCreateFolderInFolder(DriveApp, GMAIL_ATTACHMENT_FOLDER);
88
89     // Pre-load existing sub-folders so we reuse rather than duplicate them.
90     const existingFolders = new Map<string, Folder>();
91     const currentFolders = head.getFolders();
92     while (currentFolders.hasNext()) {
93         const folder = currentFolders.next();
94         existingFolders.set(folder.getName(), folder);
95     }
96     logDebug("Already present folders: " + existingFolders.size);
97
98     const addedFiles = new Map<string, Folder>();
99     const processedIds: string[] = [];
100     let addMore = maxToAdd > 0;
101     let hasError = false;

```

```

102     let errors = 0;
103     let lastError = "";
104
105     for (const thread of threads) {
106         if (!addMore) break;
107         for (const message of thread.getMessages()) {
108             if (!addMore) break;
109             const messageId = message.getId();
110             // Skip messages we fully handled on a previous run.
111             if (alreadyProcessed.has(messageId)) {
112                 logDebug("Skipping already-processed message " + messageId);
113                 continue;
114             }
115             let messageFailed = false;
116             const from = message.getFrom();
117             // GmailMessage#getDate is typed as the stripped Base.Date interface but is
118             // a real JS Date at runtime.
119             const date = message.getDate() as unknown as Date;
120             const attachments = message.getAttachments();
121             let lastHandled = -1;
122             for (let a = 0; a < attachments.length && addMore; a++) {
123                 const attachment = attachments[a];
124                 lastHandled = a;
125                 const contentType = attachment.getContentType();
126                 const fileName = getAttachmentName(attachment.getName(), from, date);
127
128                 if (!isAllowedContentType(contentType) || addedFiles.has(fileName)) {
129                     logDebug("Rejected file of type: " + contentType);
130                     continue;
131                 }
132
133                 let folder = resolveFolder(
134                     head,
135                     existingFolders,
136                     structure,
137                     contentType,
138                     date,
139                 );
140                 if (folder == null) {
141                     // Unknown structure: drop it in the top-level folder for recovery.
142                     logError("Could not resolve directory; using top-level folder.");
143                     folder = head;
144                 }
145
146                 try {
147                     // Skip if a file with this name already exists in the folder.
148                     if (folder.GetFilesByName(fileName).hasNext()) {
149                         continue;
150                     }
151                     const blob = attachment.copyBlob();
152                     blob.setName(fileName);
153                     const file = folder.createFile(blob);
154                     file.setDescription(
155                         "Created from a Gmail Attachment from " +
156                         from +
157                         " sent on date " +
158                         date,
159                     );
160                     addedFiles.set(fileName, folder);
161                     Utilities.sleep(100);
162                     if (addedFiles.size === maxToAdd) {
163                         addMore = false;
164                     }
165                 } catch (e) {
166                     logError("Attachment copy failed (message " + messageId + "): " + e);

```

```

167         hasError = true;
168         messageFailed = true;
169         errors++;
170         lastError = String(e);
171         Utilities.sleep(1000);
172     }
173 }
174 // Mark the message done only if we handled all its attachments cleanly.
175 // If we stopped early on the daily cap (lastHandled short of the end) or
176 // hit a transient failure, leave it unmarked so the next run retries it.
177 const completed = !messageFailed && lastHandled === attachments.length - 1;
178 if (completed) {
179     processedIds.push(messageId);
180 }
181 }
182 }
183
184 return { addedFiles: addedFiles.size, hasError, errors, lastError, processedIds };
185 }
186

```

8. user (2026-06-10T17:59:59.049Z)

```

1  import {
2      PROP_DEBUG,
3      PROP_FILES_ADDED_TODAY,
4      PROP_FOLDER,
5      PROP_HISTORY_ID,
6      PROP_LAST_ERROR,
7      PROP_LAST_SYNCED,
8      PROP_PROCESSED_IDS,
9      PROP_RUN_STATS,
10     PROP_SUMMARY_LAST_SENT,
11 } from "./config";
12 import { parseProcessedIds } from "./processed";
13 import { parseStats, parseStatsRaw, type RunStats } from "./stats";
14 import type { FilesAddedToday } from "./quota";
15
16 /**
17  * Thin wrappers over PropertiesService. All persistent sync state lives in the
18  * user properties store. Replaces the removed `UserProperties` global and the
19  * deprecated `Utilities.jsonParse/jsonStringify` helpers.
20  */
21 function props(): GoogleAppsScript.Properties.Properties {
22     return PropertiesService.getUserProperties();
23 }
24
25 export function getFolderStructure(): string {
26     return props().getProperty(PROP_FOLDER) ?? "";
27 }
28
29 export function setFolderStructure(structure: string): void {
30     props().setProperty(PROP_FOLDER, structure);
31 }
32
33 export function getLastSynced(): string {
34     return props().getProperty(PROP_LAST_SYNCED) ?? "";
35 }
36
37 export function setLastSynced(iso: string): void {
38     props().setProperty(PROP_LAST_SYNCED, iso);
39 }
40
41 export function getDebug(): string | null {
42     return props().getProperty(PROP_DEBUG);
43 }

```

```

43     }
44
45     export function setDebug(value: string): void {
46         props().setProperty(PROP_DEBUG, value);
47     }
48
49     export function getFilesAddedToday(): FilesAddedToday {
50         const raw = props().getProperty(PROP_FILES_ADDED_TODAY);
51         if (!raw) {
52             return { added: 0, date: new Date().toJSON() };
53         }
54         return JSON.parse(raw) as FilesAddedToday;
55     }
56
57     export function setFilesAddedToday(value: FilesAddedToday): void {
58         props().setProperty(PROP_FILES_ADDED_TODAY, JSON.stringify(value));
59     }
60
61     /** The set of Gmail message ids we have already processed. */
62     export function getProcessedIds(): string[] {
63         return parseProcessedIds(props().getProperty(PROP_PROCESSED_IDS));
64     }
65
66     export function setProcessedIds(ids: string[]): void {
67         props().setProperty(PROP_PROCESSED_IDS, JSON.stringify(ids));
68     }
69
70     /** Per-day run statistics accumulator (fresh for `day` if absent/stale). */
71     export function getRunStats(day: string): RunStats {
72         return parseStats(props().getProperty(PROP_RUN_STATS), day);
73     }
74
75     /** The persisted run stats as-is (may belong to a prior day), or null. */
76     export function getRunStatsRaw(): RunStats | null {
77         return parseStatsRaw(props().getProperty(PROP_RUN_STATS));
78     }
79
80     export function setRunStats(stats: RunStats): void {
81         props().setProperty(PROP_RUN_STATS, JSON.stringify(stats));
82     }
83
84     /** The day key (YYYY-MM-DD) of the last summary we sent, or null. */
85     export function getSummaryLastSent(): string | null {
86         return props().getProperty(PROP_SUMMARY_LAST_SENT);
87     }
88
89     export function setSummaryLastSent(day: string): void {
90         props().setProperty(PROP_SUMMARY_LAST_SENT, day);
91     }
92
93     /**
94      * The Gmail `historyId` from which incremental sync should resume, or null when
95      * we are still backfilling (date-window mode) or the id has expired.
96      */
97     export function getHistoryId(): string | null {
98         return props().getProperty(PROP_HISTORY_ID);
99     }
100
101     export function setHistoryId(id: string): void {
102         props().setProperty(PROP_HISTORY_ID, id);
103     }
104
105     /** Drops the stored historyId, forcing the next run back into backfill mode. */
106     export function clearHistoryId(): void {
107         props().deleteProperty(PROP_HISTORY_ID);

```

```

108     }
109
110     /** Records the last execution error so it can be inspected later. */
111     export function setLastError(message: string): void {
112         props().setProperty(
113             PROP_LAST_ERROR,
114             JSON.stringify({ message, at: new Date().toJSON() }),
115         );
116     }
117
118     /** Wipes all persisted state. Drive files are unaffected. */
119     export function clearAll(): void {
120         props().deleteAllProperties();
121     }
122

```

9. user (2026-06-10T17:59:59.215Z)

```

1     /**
2      * Pure helpers for the set of already-processed Gmail message IDs. Keeping this
3      * logic free of any Google service makes it directly unit-testable.
4      *
5      * The set is bounded: PropertiesService caps a single value at 9 KB, and a
6      * message ID is ~16 chars, so we keep the most recent MAX_PROCESSED_IDS ids in
7      * insertion order (oldest dropped first). Re-scanning an old window may then
8      * re-probe a long-evicted message, which is harmless ? the Drive
9      * `getFilesByName` check still prevents a duplicate copy.
10     */
11
12     /** Upper bound on retained ids; ~16 chars each keeps us well under the 9 KB cap. */
13     export const MAX_PROCESSED_IDS = 400;
14
15     /** Parses the persisted JSON array, tolerating null/garbage as "empty". */
16     export function parseProcessedIds(raw: string | null): string[] {
17         if (!raw) {
18             return [];
19         }
20         try {
21             const parsed = JSON.parse(raw);
22             return Array.isArray(parsed) ? (parsed as string[]) : [];
23         } catch {
24             return [];
25         }
26     }
27
28     /**
29      * Returns a new list with `id` appended (if not already present), trimmed to
30      * the newest `max` entries. Existing ids are not reordered.
31     */
32     export function addProcessedId(
33         ids: string[],
34         id: string,
35         max: number = MAX_PROCESSED_IDS,
36     ): string[] {
37         if (ids.includes(id)) {
38             return ids;
39         }
40         const next = ids.concat(id);
41         return next.length > max ? next.slice(next.length - max) : next;
42     }
43

```

10. user (2026-06-10T18:00:04.251Z)

```

1     /**

```

```

2      * The MIT License (MIT)
3      * Copyright (c) 2013 Avi Dullu
4      */
5
6      /** Top-level Drive folder that holds every synced attachment. */
7      export const GMAIL_ATTACHMENT_FOLDER = "GmailAttachments";
8
9      /** Public project / support page shown in the setup UI. */
10     export const SUPPORT_URL =
11         "https://github.com/avidullu/gmail-attachments-to-gdrive";
12
13     /** Privacy policy shown in the setup UI (and the OAuth consent screen). */
14     export const PRIVACY_POLICY_URL =
15         "https://github.com/avidullu/gmail-attachments-to-gdrive/blob/master/PRIVACY.md";
16
17     /**
18      * Apps Script enforces a daily quota on file creation. We cap ourselves below
19      * it so a single run can never exhaust the account's allowance.
20      */
21     export const MAX_FILES_ADD_PER_DAY = 240;
22
23     // Time constants, in milliseconds.
24     export const ONE_MINUTE = 60_000;
25     export const ONE_DAY = 86_400_000;
26     export const THIRTY_DAYS = 2_592_000_000;
27     export const ONE_YEAR = 31_622_400_000;
28     export const SIX_HOURS = 21_600_000;
29
30     /**
31      * Wall-clock budget for a single sync execution. Apps Script kills consumer
32      * scripts at 6 minutes, so we stop at 5 to leave margin for the final
33      * bookkeeping and trigger scheduling.
34      */
35     export const MAX_RUNTIME_MS = 5 * ONE_MINUTE;
36
37     /** Defensive upper bound on how many sync triggers may exist at once (limit is 20). */
38     export const MAX_SYNC_TRIGGERS = 12;
39
40     // Keys used in PropertiesService.getUserProperties().
41     export const PROP_FOLDER = "f";
42     export const PROP_LAST_SYNCED = "l";
43     export const PROP_FILES_ADDED_TODAY = "fa";
44     export const PROP_DEBUG = "dbg";
45     export const PROP_LAST_ERROR = "err";
46     export const PROP_PROCESSED_IDS = "pid";
47     export const PROP_RUN_STATS = "stats";
48     export const PROP_SUMMARY_LAST_SENT = "sumsent";
49     export const PROP_HISTORY_ID = "hid";
50
51     /** Flip to true to emit verbose Logger output during development. */
52     export const ENABLE_DEBUGGING = false;
53
54     /**
55      * Hybrid incremental sync. OFF by default so the proven date-window backfill
56      * remains the sole code path; flip to true to roll out the cheaper Gmail
57      * `history.list` steady state. When on, once a backfill catches up the current
58      * mailbox `historyId` is captured and subsequent runs process only newly-added
59      * messages, falling back to a date-window re-scan if the id expires (HTTP 404).
60      * Requires the Gmail advanced service (declared in appsscript.json); no extra
61      * OAuth scope ? `gmail.readonly` already covers history reads.
62      */
63     export const ENABLE_INCREMENTAL_SYNC = false;
64
65     /**
66      * Daily summary email. OFF by default ? enabling it requires the

```

```

67     * `script.send_mail` OAuth scope (a re-consent on next deploy). When enabled,
68     * set SUMMARY_EMAIL_RECIPIENT to the destination address (empty = the
69     * effective user's own address).
70     */
71     export const ENABLE_SUMMARY_EMAIL = false;
72     export const SUMMARY_EMAIL_RECIPIENT = "";
73

```

11. user (2026-06-10T18:00:04.681Z)

```

1     import { ONE_DAY, THIRTY_DAYS } from "./config";
2
3     export interface SearchQuery {
4         /** The Gmail search expression. */
5         query: string;
6         /** ISO timestamp of the upper bound of this window (for debugging). */
7         before: string;
8         /** True once the window has reached "now" ? i.e. we are fully caught up. */
9         isToday: boolean;
10    }
11
12    /**
13     * Builds the Gmail search for the next 30-day window after `lastSynced`.
14     *
15     * `now` is injected (rather than read from the clock) so the windowing logic is
16     * deterministic under test. NOTE: this intentionally preserves the legacy
17     * behaviour of mixing a UTC upper bound (`Date.parse`) with local-time day
18     * parts (`getFullYear`/`getMonth`/`getDate`) when formatting the Gmail
19     * `before:`/`after:` strings, so the day boundaries follow the script's
20     * configured timezone.
21     */
22    export function buildSearchQuery(lastSynced: string, now: Date): SearchQuery {
23        let query = "to:me in:all has:attachment -in:trash -in:spam ";
24        let nextDate = THIRTY_DAYS + Date.parse(lastSynced);
25        const today = now.getTime();
26        let isToday = false;
27        if (today < nextDate) {
28            isToday = true;
29            nextDate = today;
30        }
31        nextDate += ONE_DAY;
32
33        const afterDate = new Date(Date.parse(lastSynced) - ONE_DAY);
34        const beforeDate = new Date(nextDate);
35        const beforeQ = `${beforeDate.getFullYear()}/${beforeDate.getMonth() +
1}/${beforeDate.getDate()}`;
36        const afterQ = `${afterDate.getFullYear()}/${afterDate.getMonth() +
1}/${afterDate.getDate()}`;
37
38        if (beforeQ === afterQ) {
39            // Window collapses to a single day; just search "after yesterday".
40            const yesterday = new Date(Date.parse(lastSynced) - ONE_DAY);
41            const yesterdayQ = `${yesterday.getFullYear()}/${yesterday.getMonth() +
1}/${yesterday.getDate()}`;
42            query = query + " " + "after:" + yesterdayQ;
43        } else {
44            query = query + "before:" + beforeQ + " " + "after:" + afterQ;
45        }
46
47        return { query, before: beforeDate.toJSON(), isToday };
48    }
49

```

12. user (2026-06-10T18:00:04.917Z)

```

1      {
2        "timeZone": "America/Los_Angeles",
3        "runtimeVersion": "V8",
4        "exceptionLogging": "STACKDRIVER",
5        "dependencies": {
6          "enabledAdvancedServices": [
7            {
8              "userSymbol": "Gmail",
9              "serviceId": "gmail",
10             "version": "v1"
11            }
12          ]
13        },
14        "oauthScopes": [
15          "https://www.googleapis.com/auth/gmail.readonly",
16          "https://www.googleapis.com/auth/drive",
17          "https://www.googleapis.com/auth/script.scriptapp"
18        ],
19        "webapp": {
20          "access": "ANYONE",
21          "executeAs": "USER_ACCESSING"
22        }
23      }
24

```

13. user (2026-06-10T18:00:09.564Z)

```

1      import {
2        ENABLE_SUMMARY_EMAIL,
3        SUMMARY_EMAIL_RECIPIENT,
4      } from "./config";
5      import { logError, logInfo } from "./logger";
6      import { dayKey, formatSummary } from "./stats";
7      import * as state from "./state";
8
9      /**
10     * Summarises the previous day's activity the first time a sync runs on a new
11     * calendar day. Always logs the summary; emailing is opt-in
12     * (ENABLE_SUMMARY_EMAIL, which requires the script.send_mail scope).
13     *
14     * `now` is injected for testability.
15     */
16     export function maybeSendDailySummary(now: Date): void {
17       const today = dayKey(now);
18       if (state.getSummaryLastSent() === today) {
19         return; // already summarised since the last day rolled over
20       }
21
22       // Summarise whatever the persisted accumulator holds (the prior day's run).
23       const stats = state.getRunStatsRaw();
24       if (!stats || stats.day === today) {
25         // Nothing from a prior day to report yet; arm for the next rollover.
26         state.setSummaryLastSent(today);
27         return;
28       }
29
30       const summary = formatSummary(stats);
31       logInfo(summary);
32
33       if (ENABLE_SUMMARY_EMAIL) {
34         try {
35           const to = SUMMARY_EMAIL_RECIPIENT || Session.getEffectiveUser().getEmail();
36           if (to) {
37             MailApp.sendEmail(to, "Gmail?Drive sync summary " + stats.day, summary);
38           }
39         }
40       }
41     }

```

```

39         } catch (e) {
40             logError("Failed to send summary email: " + e);
41         }
42     }
43
44     state.setSummaryLastSent(today);
45 }
46

```

14. user (2026-06-10T18:00:10.114Z)

```

1  /**
2   * Pure helpers for per-day run statistics. The accumulator is persisted as JSON
3   * in user properties; keeping the math here (no Google services) makes it
4   * directly unit-testable.
5   */
6
7  export interface RunStats {
8      /** Calendar day these stats belong to (YYYY-MM-DD, local time). */
9      day: string;
10     /** Number of sync passes that ran on this day. */
11     runs: number;
12     /** Files copied to Drive on this day. */
13     filesAdded: number;
14     /** Attachment/message errors encountered on this day. */
15     errors: number;
16     /** Most recent error message of the day, if any. */
17     lastError: string;
18 }
19
20 /** Local-time YYYY-MM-DD key for a date. */
21 export function dayKey(date: Date): string {
22     const y = date.getFullYear();
23     const m = String(date.getMonth() + 1).padStart(2, "0");
24     const d = String(date.getDate()).padStart(2, "0");
25     return `${y}-${m}-${d}`;
26 }
27
28 export function emptyStats(day: string): RunStats {
29     return { day, runs: 0, filesAdded: 0, errors: 0, lastError: "" };
30 }
31
32 /** Parses persisted stats as-is, or null if absent/malformed. Day is not coerced. */
33 export function parseStatsRaw(raw: string | null): RunStats | null {
34     if (!raw) {
35         return null;
36     }
37     try {
38         const parsed = JSON.parse(raw) as Partial<RunStats>;
39         if (!parsed || typeof parsed.day !== "string") {
40             return null;
41         }
42         return {
43             day: parsed.day,
44             runs: parsed.runs ?? 0,
45             filesAdded: parsed.filesAdded ?? 0,
46             errors: parsed.errors ?? 0,
47             lastError: parsed.lastError ?? "",
48         };
49     } catch {
50         return null;
51     }
52 }
53
54 /** Parses persisted stats, returning a fresh accumulator for `day` if absent/stale. */

```

```

55 export function parseStats(raw: string | null, day: string): RunStats {
56     const parsed = parseStatsRaw(raw);
57     return parsed && parsed.day === day ? parsed : emptyStats(day);
58 }
59
60 export interface RunOutcome {
61     filesAdded: number;
62     errors: number;
63     lastError?: string;
64 }
65
66 /** Folds a single run's outcome into the day's accumulator (returns a new object). */
67 export function recordRun(stats: RunStats, outcome: RunOutcome): RunStats {
68     return {
69         day: stats.day,
70         runs: stats.runs + 1,
71         filesAdded: stats.filesAdded + outcome.filesAdded,
72         errors: stats.errors + outcome.errors,
73         lastError: outcome.lastError ? outcome.lastError : stats.lastError,
74     };
75 }
76
77 /** Human-readable one-screen summary used for logs and the optional email. */
78 export function formatSummary(stats: RunStats): string {
79     const lines = [
80         `Gmail?Drive sync summary for ${stats.day}`,
81         `  runs:      ${stats.runs}`,
82         `  files added: ${stats.filesAdded}`,
83         `  errors:     ${stats.errors}`,
84     ];
85     if (stats.lastError) {
86         lines.push(`  last error: ${stats.lastError}`);
87     }
88     return lines.join("\n");
89 }
90

```

15. user (2026-06-10T18:00:10.254Z)

```

1   import { ENABLE_DEBUGGING } from "./config";
2
3   /**
4    * Minimal structured logger. Each line is prefixed with a level so the Apps
5    * Script execution log / Stackdriver can be filtered (e.g. `severity:ERROR`).
6    * Kept dependency-free and side-effect-only so callers stay simple.
7    */
8   function emit(level: "DEBUG" | "INFO" | "ERROR", data: unknown): void {
9       Logger.log([" " + level + " " + String(data)]);
10  }
11
12  /** Verbose log, only emitted when ENABLE_DEBUGGING is on. */
13  export function logDebug(data: unknown): void {
14      if (ENABLE_DEBUGGING) {
15          emit("DEBUG", data);
16      }
17  }
18
19  /** Informational log; always emitted. */
20  export function logInfo(data: unknown): void {
21      emit("INFO", data);
22  }
23
24  /** Error log; always emitted and surfaces in Stackdriver as an error. */
25  export function logError(data: unknown): void {
26      emit("ERROR", data);

```

```
27     }
28
```

16. user (2026-06-10T18:00:17.165Z)

```
1     import {
2         MAX_FILES_ADD_PER_DAY,
3         ONE_DAY,
4         PROP_DEBUG,
5         PROP_FILES_ADDED_TODAY,
6         PROP_FOLDER,
7         PROP_LAST_SYNCED,
8         PROP_PROCESSED_IDS,
9     } from "../src/config";
10    import { syncUserDrives } from "../src/sync";
11
12    type Store = Record<string, string>;
13
14    function fakeProps(initial: Store) {
15        const store: Store = { ...initial };
16        return {
17            store,
18            getProperty: (k: string) => (k in store ? store[k] : null),
19            setProperty: (k: string, v: string) => {
20                store[k] = String(v);
21            },
22            deleteAllProperties: () => {
23                for (const k of Object.keys(store)) delete store[k];
24            },
25        };
26    }
27
28    interface FakeTrigger {
29        getHandlerFunction: () => string;
30    }
31
32    function fakeScriptApp() {
33        const triggers: FakeTrigger[] = [];
34        return {
35            triggers,
36            newTrigger: (handler: string) => ({
37                timeBased: () => ({
38                    after: () => ({
39                        create: () => {
40                            triggers.push({ getHandlerFunction: () => handler });
41                        },
42                    }),
43                }),
44            }),
45            getProjectTriggers: () => [...triggers],
46            deleteTrigger: (t: FakeTrigger) => {
47                const i = triggers.indexOf(t);
48                if (i >= 0) triggers.splice(i, 1);
49            },
50        };
51    }
52
53    function fakeLock(canLock: boolean) {
54        const releaseLock = jest.fn();
55        return {
56            releaseLock,
57            service: {
58                getUserLock: () => ({ tryLock: () => canLock, releaseLock }),
59            },
60        };
61    }
```

```

61     }
62
63     const g = globalThis as unknown as Record<string, unknown>;
64     const nowIso = () => new Date().toJSON();
65
66     function install(props: ReturnType<typeof fakeProps>, lockable = true) {
67         const script = fakeScriptApp();
68         const lock = fakeLock(lockable);
69         g.PropertiesService = { getUserProperties: () => props };
70         g.ScriptApp = script;
71         g.LockService = lock.service;
72         g.GmailApp = { search: jest.fn(() => []) };
73         return { script, lock };
74     }
75
76     function baseProps(overrides: Store = {}) {
77         return fakeProps({
78             [PROP_DEBUG]: "1",
79             [PROP_FOLDER]: "byContentType",
80             [PROP_LAST_SYNCED]: nowIso(),
81             [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
82             ...overrides,
83         });
84     }
85
86     describe("syncUserDrives", () => {
87         it("skips entirely when the lock is held by another run", () => {
88             const props = baseProps();
89             const { script } = install(props, /* lockable */ false);
90
91             syncUserDrives();
92
93             expect(script.triggers).toHaveLength(0);
94             expect(g.GmailApp).toBeDefined();
95             expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
96         });
97
98         it("replaces the previous trigger generation and schedules new ones", () => {
99             const props = baseProps();
100             const { script, lock } = install(props);
101             // Seed a stale trigger from a previous run.
102             const stale = { getHandlerFunction: () => "syncUserDrives" };
103             script.triggers.push(stale);
104
105             syncUserDrives();
106
107             expect(script.triggers).not.toContain(stale);
108             // Fan-out (6) + tail (20m) created, capped well under the 20 limit.
109             expect(script.triggers.length).toBeGreaterThanOrEqual(6);
110             expect(script.triggers.length).toBeLessThanOrEqual(12);
111             expect(lock.releaseLock).toHaveBeenCalled();
112         });
113
114         it("does not search Gmail when the daily cap is already reached", () => {
115             const props = baseProps({
116                 [PROP_FILES_ADDED_TODAY]: JSON.stringify({
117                     added: MAX_FILES_ADD_PER_DAY,
118                     date: nowIso(),
119                 }),
120             });
121             install(props);
122
123             syncUserDrives();
124
125             expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();

```

```

126     });
127
128     it("resets the daily counter on a new calendar day", () => {
129         const yesterday = new Date(Date.now() - ONE_DAY).toJSON();
130         const props = baseProps({
131             [PROP_FILES_ADDED_TODAY]: JSON.stringify({
132                 added: MAX_FILES_ADD_PER_DAY,
133                 date: yesterday,
134             }),
135         });
136         install(props);
137
138         syncUserDrives();
139
140         // Counter was reset, so the sync loop ran and persisted added: 0.
141         expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
142         const persisted = JSON.parse(props.store[PROP_FILES_ADDED_TODAY]);
143         expect(persisted.added).toBe(0);
144     });
145
146     it("persists the ids of messages it copied for cross-run dedup", () => {
147         const props = baseProps();
148         install(props);
149
150         // A minimal Drive + Gmail wiring so one attachment is copied.
151         const iter = <T>(items: T[]) => {
152             let i = 0;
153             return { hasNext: () => i < items.length, next: () => items[i++] };
154         };
155         const head = {
156             getName: () => "GmailAttachments",
157             getFolders: () => iter([]),
158             createFolder: () => head,
159             getFilesByName: () => iter([]),
160             createFile: () => ({ setDescription: () => undefined }),
161         };
162         g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };
163         const message = {
164             getId: () => "msg-xyz",
165             getFrom: () => "alice@example.com",
166             getDate: () => new Date(2012, 5, 6),
167             getAttachments: () => [
168                 {
169                     getContentType: () => "application/pdf",
170                     getName: () => "report.pdf",
171                     copyBlob: () => ({ setName: () => undefined }),
172                 },
173             ],
174         };
175         let firstCall = true;
176         (g.GmailApp as { search: jest.Mock }).search = jest.fn(() => {
177             // Return the thread once, then nothing so the loop converges.
178             if (firstCall) {
179                 firstCall = false;
180                 return [{ getMessages: () => [message] }];
181             }
182             return [];
183         });
184
185         syncUserDrives();
186
187         expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("msg-xyz");
188     });
189 });
190

```

17. user (2026-06-10T18:00:17.403Z)

```
1      // Enable the opt-in incremental path for this suite only.
2      jest.mock("../src/config", () => ({
3          ...jest.requireActual("../src/config"),
4          ENABLE_INCREMENTAL_SYNC: true,
5      }));
6
7      import {
8          ONE_DAY,
9          PROP_DEBUG,
10         PROP_FILES_ADDED_TODAY,
11         PROP_FOLDER,
12         PROP_HISTORY_ID,
13         PROP_LAST_SYNCED,
14         PROP_PROCESSED_IDS,
15     } from "../src/config";
16     import { syncUserDrives } from "../src/sync";
17
18     type Store = Record<string, string>;
19     const g = globalThis as unknown as Record<string, unknown>;
20     const nowIso = () => new Date().toJSON();
21
22     function fakeProps(initial: Store) {
23         const store: Store = { ...initial };
24         return {
25             store,
26             getProperty: (k: string) => (k in store ? store[k] : null),
27             setProperty: (k: string, v: string) => {
28                 store[k] = String(v);
29             },
30             deleteProperty: (k: string) => {
31                 delete store[k];
32             },
33             deleteAllProperties: () => {
34                 for (const k of Object.keys(store)) delete store[k];
35             },
36         };
37     }
38
39     function fakeScriptApp() {
40         const triggers: { getHandlerFunction: () => string }[] = [];
41         return {
42             newTrigger: (handler: string) => ({
43                 timeBased: () => ({
44                     after: () => ({
45                         create: () => triggers.push({ getHandlerFunction: () => handler }),
46                     }),
47                 }),
48             }),
49             getProjectTriggers: () => [...triggers],
50             deleteTrigger: () => undefined,
51         };
52     }
53
54     function iter<T>(items: T[]) {
55         let i = 0;
56         return { hasNext: () => i < items.length, next: () => items[i++] };
57     }
58
59     function installBase(props: ReturnType<typeof fakeProps>) {
60         g.PropertiesService = { getUserProperties: () => props };
61         g.ScriptApp = fakeScriptApp();
62         g.LockService = {
63             getUserLock: () => ({ tryLock: () => true, releaseLock: jest.fn() }),
64         };
65     }
```

```

64     };
65     g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
66 }
67
68 function baseProps(overrides: Store = {}) {
69     return fakeProps({
70         [PROP_DEBUG]: "1",
71         [PROP_FOLDER]: "byContentType",
72         [PROP_LAST_SYNCED]: nowIso(),
73         [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
74         ...overrides,
75     });
76 }
77
78 describe("syncUserDrives ? incremental mode", () => {
79     it("processes messages added since the stored historyId and advances it", () => {
80         const props = baseProps({ [PROP_HISTORY_ID]: "100" });
81         installBase(props);
82
83         // A Drive head folder that accepts one file.
84         const head = {
85             getName: () => "GmailAttachments",
86             getFolders: () => iter([]),
87             createFolder: () => head,
88             getFilesByName: () => iter([]),
89             createFile: () => ({ setDescription: () => undefined }),
90         };
91         g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };
92
93         const message = {
94             getId: () => "m1",
95             getFrom: () => "alice@example.com",
96             getDate: () => new Date(2012, 5, 6),
97             getAttachments: () => [
98                 {
99                     getContentType: () => "application/pdf",
100                     getName: () => "report.pdf",
101                     copyBlob: () => ({ setName: () => undefined }),
102                 },
103             ],
104         };
105         (g.GmailApp as { getMessageById: jest.Mock }).getMessageById = jest.fn(
106             () => ({ getThread: () => ({ getId: () => "t1", getMessages: () => [message] })
107         });
108         g.Gmail = {
109             Users: {
110                 getProfile: jest.fn(),
111                 History: {
112                     list: jest.fn(() => ({
113                         history: [{ messagesAdded: [{ message: { id: "m1" } } ] },
114                         historyId: "200",
115                     })),
116                 },
117             },
118         };
119
120         syncUserDrives();
121
122         // Never falls back to a date-window search while the history is valid.
123         expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
124         expect(props.store[PROP_HISTORY_ID]).toBe("200");
125         expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("m1");
126     });
127

```

```

128     it("clears the cursor and rewinds lastSynced when the history expired", () => {
129         const props = baseProps({ [PROP_HISTORY_ID]: "1" });
130         installBase(props);
131         g.Gmail = {
132             Users: {
133                 getProfile: jest.fn(),
134                 History: {
135                     list: jest.fn(() => {
136                         throw new Error("call failed with error: 404 not found");
137                     }),
138                 },
139             },
140         };
141
142         syncUserDrives();
143
144         expect(PROP_HISTORY_ID in props.store).toBe(false);
145         // lastSynced rewound to roughly 30 days ago for the recovery re-scan.
146         const lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
147         expect(Date.now() - lastSynced).toBeGreaterThan(29 * ONE_DAY);
148         // No history ? no incremental processing this run; backfill resumes next run.
149         expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
150     });
151
152     it("captures the mailbox historyId once backfill catches up", () => {
153         // No stored historyId ? backfill mode; lastSynced=now means we are already
154         // caught up, so the run captures the current historyId for next time.
155         const props = baseProps();
156         installBase(props);
157         g.Gmail = {
158             Users: {
159                 getProfile: jest.fn(() => ({ historyId: "555" })),
160                 History: { list: jest.fn() },
161             },
162         };
163
164         syncUserDrives();
165
166         expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
167         expect(props.store[PROP_HISTORY_ID]).toBe("555");
168     });
169 });
170

```

18. user (2026-06-10T18:00:22.331Z)

```

1     import { updateDrive } from "../src/drive";
2
3     // ---- Minimal fakes for the Drive/Gmail objects updateDrive touches ----
4
5     function iterator<T>(items: T[]) {
6         let i = 0;
7         return { hasNext: () => i < items.length, next: () => items[i++] };
8     }
9
10    class FakeFile {
11        description = "";
12        constructor(public name: string) {}
13        setDescription(d: string) {
14            this.description = d;
15            return this;
16        }
17    }
18
19    class FakeFolder {

```

```

20     files: FakeFile[] = [];
21     subfolders: FakeFolder[] = [];
22     constructor(public name: string) {}
23     getName() {
24         return this.name;
25     }
26     getFolders() {
27         return iterator(this.subfolders);
28     }
29     createFolder(n: string) {
30         const f = new FakeFolder(n);
31         this.subfolders.push(f);
32         return f;
33     }
34     getFilesByName(n: string) {
35         return iterator(this.files.filter((f) => f.name === n));
36     }
37     createFile(blob: { _name: string }) {
38         const f = new FakeFile(blob._name);
39         this.files.push(f);
40         return f;
41     }
42 }
43
44 function fakeAttachment(name: string, contentType: string) {
45     return {
46         getName: () => name,
47         getContentType: () => contentType,
48         copyBlob: () => ({
49             _name: name,
50             setName(n: string) {
51                 this._name = n;
52                 return this;
53             },
54         })),
55     };
56 }
57
58 let nextMsgId = 0;
59 function fakeThread(
60     from: string,
61     date: Date,
62     attachments: unknown[],
63     id?: string,
64 ) {
65     const messageId = id ?? `msg-${nextMsgId++}`;
66     const message = {
67         getId: () => messageId,
68         getFrom: () => from,
69         getDate: () => date,
70         getAttachments: () => attachments,
71     };
72     return { getMessages: () => [message] };
73 }
74
75 let head: FakeFolder;
76
77 beforeEach(() => {
78     head = new FakeFolder("GmailAttachments");
79     (globalThis as unknown as Record<string, unknown>).DriveApp = {
80         getFolders: () => iterator<FakeFolder>([]),
81         createFolder: () => head,
82     };
83 });
84

```

```

85     const date = new Date(2012, 5, 6, 9, 5, 3);
86
87     describe("updateDrive", () => {
88         it("copies an allowed attachment into the right structure folder", () => {
89             const threads = [
90                 fakeThread("alice@example.com", date, [
91                     fakeAttachment("report.pdf", "application/pdf"),
92                 ]),
93             ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
94
95             const result = updateDrive(threads, "byContentType", 10);
96
97             expect(result.addedFiles).toBe(1);
98             expect(result.hasError).toBe(false);
99             const bucket = head.subfolders.find((f) => f.name === "Application Data");
100             expect(bucket?.files).toHaveLength(1);
101         });
102
103         it("skips disallowed content types", () => {
104             const threads = [
105                 fakeThread("alice@example.com", date, [
106                     fakeAttachment("evil.exe", "application/x-msdownload"),
107                 ]),
108             ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
109
110             expect(updateDrive(threads, "byContentType", 10).addedFiles).toBe(0);
111         });
112
113         it("does not copy the same file twice within a run", () => {
114             const threads = [
115                 fakeThread("alice@example.com", date, [
116                     fakeAttachment("report.pdf", "application/pdf"),
117                     fakeAttachment("report.pdf", "application/pdf"),
118                 ]),
119             ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
120
121             expect(updateDrive(threads, "byContentType", 10).addedFiles).toBe(1);
122         });
123
124         it("stops once maxToAdd files have been written", () => {
125             const threads = [
126                 fakeThread("alice@example.com", date, [
127                     fakeAttachment("a.pdf", "application/pdf"),
128                     fakeAttachment("b.pdf", "application/pdf"),
129                     fakeAttachment("c.pdf", "application/pdf"),
130                 ]),
131             ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
132
133             expect(updateDrive(threads, "byContentType", 2).addedFiles).toBe(2);
134         });
135
136         it("reports a fully-handled message id in processedIds", () => {
137             const threads = [
138                 fakeThread(
139                     "alice@example.com",
140                     date,
141                     [fakeAttachment("report.pdf", "application/pdf")],
142                     "m1",
143                 ),
144             ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
145
146             expect(updateDrive(threads, "byContentType", 10).processedIds).toEqual([
147                 "m1",
148             ]);
149         });

```

```

150
151     it("skips messages already in the processed set without touching Drive", () => {
152         const threads = [
153             fakeThread(
154                 "alice@example.com",
155                 date,
156                 [fakeAttachment("report.pdf", "application/pdf")],
157                 "m1",
158             ),
159         ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
160
161         const result = updateDrive(
162             threads,
163             "byContentType",
164             10,
165             new Set(["m1"]),
166         );
167
168         expect(result.addedFiles).toBe(0);
169         expect(result.processedIds).toEqual([]);
170         expect(head.subfolders).toHaveLength(0);
171     });
172
173     it("does not mark a message done when the daily cap truncates it", () => {
174         const threads = [
175             fakeThread(
176                 "alice@example.com",
177                 date,
178                 [
179                     fakeAttachment("a.pdf", "application/pdf"),
180                     fakeAttachment("b.pdf", "application/pdf"),
181                 ],
182                 "m1",
183             ),
184         ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
185
186         // maxToAdd = 1 copies only the first of two attachments.
187         const result = updateDrive(threads, "byContentType", 1, new Set());
188
189         expect(result.addedFiles).toBe(1);
190         expect(result.processedIds).toEqual([]);
191     });
192 });
193

```

19. user (2026-06-10T18:00:23.416Z)

```

1     import {
2         getCurrentHistoryId,
3         listAddedSince,
4         threadsForMessageIds,
5     } from "../src/gmailHistory";
6
7     const g = globalThis as unknown as Record<string, unknown>;
8
9     describe("getCurrentHistoryId", () => {
10         it("returns the profile's historyId", () => {
11             g.Gmail = {
12                 Users: { getProfile: () => ({ historyId: "99" }) }, History: { list: jest.fn() } },
13             };
14             expect(getCurrentHistoryId()).toBe("99");
15         });
16
17         it("returns null when the profile has no historyId", () => {
18             g.Gmail = {

```

```

19     Users: { getProfile: () => ({}) }, History: { list: jest.fn() } },
20 };
21 expect(getCurrentHistoryId()).toBeNull();
22 });
23 });
24
25 describe("listAddedSince", () => {
26     it("pages through results, returning collected ids and the latest historyId", () => {
27         const pages = [
28             {
29                 history: [{ messagesAdded: [{ message: { id: "m1" } }] }],
30                 historyId: "100",
31                 nextPageToken: "p2",
32             },
33             {
34                 history: [{ messagesAdded: [{ message: { id: "m2" } }] }],
35                 historyId: "200",
36             },
37         ];
38         let call = 0;
39         const list = jest.fn(() => pages[call++]);
40         g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
41
42         expect(listAddedSince("50")).toEqual({
43             messageIds: ["m1", "m2"],
44             historyId: "200",
45         });
46         expect(list).toHaveBeenCalledTimes(2);
47     });
48
49     it("returns null when the start historyId has expired", () => {
50         const list = jest.fn(() => {
51             throw new Error("call failed with error: 404 not found");
52         });
53         g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
54         expect(listAddedSince("1")).toBeNull();
55     });
56
57     it("rethrows errors that are not history expiry", () => {
58         const list = jest.fn(() => {
59             throw new Error("backend error 500");
60         });
61         g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
62         expect(() => listAddedSince("1")).toThrow("500");
63     });
64 });
65
66 describe("threadsForMessageIds", () => {
67     it("resolves distinct threads and skips messages that no longer exist", () => {
68         const threadA = { getId: () => "tA" };
69         const byId: Record<string, { getThread: () => { getId: () => string } }> = {
70             a: { getThread: () => threadA },
71             b: { getThread: () => threadA }, // same thread as a
72             c: { getThread: () => ({ getId: () => "tC" }) },
73         };
74         g.GmailApp = { getMessageById: (id: string) => byId[id] ?? null };
75
76         const threads = threadsForMessageIds(["a", "b", "c", "gone"]);
77         expect(threads.map((t) => (t as { getId: () => string }).getId()).sort()).toEqual(
78             ["tA", "tC"],
79         );
80     });
81 });
82

```

20. user (2026-06-10T18:00:23.661Z)

```
1  import {
2    collectAddedMessageIds,
3    isHistoryExpired,
4    nextHistoryId,
5    type HistoryListResponse,
6  } from "../src/historySync";
7
8  describe("collectAddedMessageIds", () => {
9    it("collects unique ids across pages in first-seen order", () => {
10      const responses: HistoryListResponse[] = [
11        {
12          history: [
13            { messagesAdded: [{ message: { id: "a" } }, { message: { id: "b" } }] },
14          ],
15        },
16        {
17          history: [
18            { messagesAdded: [{ message: { id: "b" } }, { message: { id: "c" } }] },
19          ],
20        },
21      ];
22      expect(collectAddedMessageIds(responses)).toEqual(["a", "b", "c"]);
23    });
24
25    it("ignores records without messagesAdded and entries without an id", () => {
26      const responses: HistoryListResponse[] = [
27        { history: [{}, { messagesAdded: [{}, { message: {} }] } ] },
28        {},
29      ];
30      expect(collectAddedMessageIds(responses)).toEqual([]);
31    });
32  });
33
34  describe("nextHistoryId", () => {
35    it("returns the last page's historyId", () => {
36      expect(nextHistoryId([{ historyId: "10" }, { historyId: "20" }], "1")).toBe(
37        "20",
38      );
39    });
40
41    it("skips trailing pages that report no historyId", () => {
42      expect(nextHistoryId([{ historyId: "10" }, {}], "1")).toBe("10");
43    });
44
45    it("falls back to the start id when no page reports one", () => {
46      expect(nextHistoryId([], {}), "7")).toBe("7");
47    });
48  });
49
50  describe("isHistoryExpired", () => {
51    it("detects a 404 details code", () => {
52      expect(isHistoryExpired({ details: { code: 404 } })).toBe(true);
53    });
54
55    it("detects 404 in the error message text", () => {
56      expect(isHistoryExpired(new Error("call failed with error: 404"))).toBe(
57        true,
58      );
59    });
60
61    it("detects a 'not found' message", () => {
62      expect(isHistoryExpired(new Error("Requested entity was not found."))).toBe(
63        true,
```

```

64     );
65   });
66
67   it("returns false for unrelated errors", () => {
68     expect(isHistoryExpired(new Error("rate limit exceeded (429)"))).toBe(false);
69     expect(isHistoryExpired(null)).toBe(false);
70   });
71 });
72

```

21. user (2026-06-10T18:00:28.904Z)

```

1    # Architecture
2
3    > Status: living document. Captures the target architecture for the
4    > extraction pivot and the decisions behind it. Product framing is in
5    > [VISION.md](./VISION.md). This supersedes the original "just copy
6    > attachments" mental model; the copier survives as one extractor at the leaf.
7
8    ## Auditability invariant (binding)
9
10   This repository is the only component that reads or processes a user's
11   Gmail, and it must remain auditable and self-deployable by a power user at any
12   time. This is the product's core privacy guarantee and a deliberate constraint
13   on every future decision (see ADR-008). It is why the project is built to be
14   open-sourced: *don't trust us ? read, and run, the code.*
15
16   Every change MUST preserve all of the following. A change that cannot is, by
17   definition, a decision to break the invariant and needs explicit owner sign-off.
18
19   1. Sole Gmail surface. Gmail is read/processed *only* here, in the user's own
20   account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ?
21   ever holds the Gmail OAuth grant or sees raw mail. Muneem may *orchestrate*
22   this app; it is never *itself* the Gmail reader.
23   2. Egress == the audited boundary. Nothing derived from a user's mail leaves
24   their Google account except through the in-repo REDACT/MINIMIZE stage,
25   carrying only minimized, non-PII signal, only to the user's own hosted
26   service, never to third parties. The set of egress destinations is explicit
27   and reviewable in source.
28   3. Run-your-own-from-source. A power user can build and deploy their own
29   instance from the published source and obtain an equivalent app; the build is
30   reproducible and the verification steps are documented ? "trust by running the
31   code you read," not merely "read the code."
32   4. No secrets in source. Credentials exist only as deploy-time secrets; the
33   repository contains nothing sensitive.
34   5. Transparent over clever at the read and egress/redaction boundaries ?
35   prefer obviously-correct, reviewable code even at some cost to brevity.
36
37   PR review checklist (apply to every change):
38
39   - [ ] Gmail access stays entirely within this repo, in-account?
40   - [ ] If any data leaves the account, does it pass through REDACT/MINIMIZE and go
41     only to the user's own service (non-PII)?
42   - [ ] Build still reproducible and self-deployable from source?
43   - [ ] No new secrets, third-party endpoints, or network calls? (Default: none ?
44     enforced by `tests/auditability.test.ts`.)
45
46   ## Principles (preserved from today's codebase)
47
48   1. Shells around pure functions. GAS-facing code is a thin shell; all logic
49   is pure and unit-tested.
50   2. Quota discipline. Respect the 6-minute execution limit, the 90-min/day
51   trigger budget, and per-user caps. Self-healing trigger fan-out; per-user
52   lock; "mark done only on clean completion."
53   3. Per-user isolation. Runs as the accessing user; state is per-user.

```

```

4. **Versioning & provenance everywhere.** Every persisted shape carries a
55 `schemaVersion`; every extracted entity carries where it came from and which
56 extractor (and version) produced it.
57
58 ## The pipeline
59
60 This repo owns the left half; downstream services own the right half. They meet
61 at the **handoff contract** only.
62
63 ```
64             ?????????????????????? this repo ?????????????????????? ?? downstream
??
65 Gmail ??? INGEST ??? CLASSIFY ??? EXTRACT ??? REDACT/MINIMIZE ??? HANDOFF ??? Muneem
66 /
67 /
68 (rich (type + (normalize (strip PII, (sink) grocery
69 /
70 EmailMsg) confidence) light) encrypt) coupon
71 svc
72 ? ??
73 parse,
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

111 // encrypted in transit; PII stripped or tokenized
112 }
113 ...
114
115 - **Entity keys** give content-stable dedup (re-mailed receipts; cross-source
116   fusion of an email confirmation + a PDF ticket ? one entity).
117 - **Attachment `sha256`** enables content-addressable dedup of re-sent files.
118 - **Provenance + `extractorVersion`** make "this extraction is stale ? re-run"
119   possible and auditable.
120
121 ## The extraction ? processing boundary
122
123 - This repo produces a normalized, classified, **redacted** `HandoffPayload` per
124   detected document/entity and routes it to a downstream **sink**.
125 - Downstream services (Muneem for finance; siblings for groceries/coupons) own
126   domain parsing, indexing, **encrypted storage**, and analytics.
127 - **Contract is owned jointly, defined from our side first.** The exact
128   Muneem-facing schema and transport are **TBD and aligned with that repo**
129   (this repo's GitHub tooling is scoped to itself, so the contract is specified
130   here and reconciled with Muneem separately).
131 - Hard rule: this repo imports **no** domain-parsing logic; downstream reads
132   **no** Gmail.
133
134 ## Storage abstraction (decision: extensible, not baked-in)
135
136 A single `Store` / `Sink` interface; code depends only on the interface.
137
138 | Impl | When | Properties |
139 | --- | --- | --- |
140 | `GoogleSheetsStore` (user-owned "LifeIndex" sheet) | v1 | in-account, queryable, zero
infra, doubles as a first dashboard |
141 | `RemoteStore` (our hosted DB) / Muneem-API sink | later | the durable system of record
|
142
143 `PropertiesService` stays only for small per-user **control state** (cursors,
144 flags, counters) ? never the entity index.
145
146 ## Privacy & egress controls
147
148 - **Egress only to our own isolated, hosted service. No third parties.**
149 - A **Redactor/Minimizer** stage runs before any egress: strip/tokenize PII,
150   keep only non-PII signal, encrypt in transit (encrypted at rest downstream).
151 - North star: continuously **shrink** what leaves while keeping the product
152   healthy.
153 - The in-account tier (Phases 0?2) has **no egress** and needs no privacy
154   change. The external tier (Phase 3) is **flag-gated + separately consented**,
155   and updates the privacy policy to name exactly what is sent and where.
156 - **Cascade to be aware of:** sending restricted-scope Gmail data outside Google
157   is what makes Google's **CASA security assessment mandatory for a public
158   launch**. Deferred ? consistent with "stay <100 until the user base justifies
159   the investment."
160 - **Open tension to resolve (Phase 3 design):** bank-statement and receipt
161   signal (merchant, amounts, account identifiers) is often itself sensitive.
162   "Non-PII only" needs a concrete per-vertical definition of what is sent vs.
163   tokenized vs. kept fully in-account.
164
165 ## State & cursors
166
167 - Keep the hybrid `history.list` + date-window skeleton; **extend, don't
168   duplicate.**
169 - Add an **extraction-aware cursor** ("fully extracted up to X with
170   extractor-set V") distinct from the copy cursor, plus per-message extraction
171   status ? so a "re-extract window for new rules" reprocesses without
172   re-copying (content hash) or duplicating entities (entity keys).
173

```

```

174  ## Quota, cost & pacing
175
176  - **Separate budgets:** raw-copy cap (existing) vs. extraction cap vs.
177    egress/handoff cap.
178  - Per-item time budgets, **prioritization** (recent + high-confidence/trusted
179    senders first), and **deferral** of heavy items.
180  - Heavy ML/PDF work happens **downstream**, not in Apps Script ? GAS stays the
181    lightweight, reliable watcher/archiver/handoff.
182
183  ## Observability
184
185  - Per-**classifier** and per-**extractor** success / failure / ambiguity rates ?
186    not just "files added."
187  - Surfaced **review queues:** "matched no extractor" and "low confidence."
188  - Structured logs carrying `messageId`; extraction & handoff metrics.
189
190  ## Platform spectrum
191
192  1. **Pure-GAS conservative extractors** (regex, known formats `.ics`/`.csv`,
193    native Drive OCR via Doc conversion) for the easy ~70%, **in-account**.
194  2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction,
195    storage, and analytics ? fed only the minimized, non-PII handoff.
196
197  ## Decision log
198
199  | # | Decision | Rationale |
200  | --- | --- | --- |
201  | ADR-001 | Pipeline (ingest ? classify ? extract ? redact ? handoff), not a copy loop |
  Treat mail as structured data; make extraction re-runnable over a durable record |
202  | ADR-002 | Extraction (this repo) is decoupled from processing (Muneem & siblings) |
  Isolate domain logic + encrypted storage + analytics downstream; clean contract seam |
203  | ADR-003 | Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later |
  Extensible by design; never baked into the code |
204  | ADR-004 | Egress: non-PII only, to our own isolated hosted service, no third parties;
  scrub + encrypt; minimize | Privacy north star; preserves trust; in-account baseline needs no
  egress |
205  | ADR-005 | Audience: <100 (Testing) now, public later; CASA deferred | Harden + learn
  value before investing in assessment/infra |
206  | ADR-006 | First verticals: grocery receipts + bank statements (? Muneem) | Volume +
  the high-fidelity handoff path |
207  | ADR-007 | Preserve current strengths (pure/shell, quotas, triggers, per-user,
  versioning/provenance) | The recent rewrite is exactly why this pivot is feasible |
208  | ADR-008 | **Option A:** this repo is the sole Gmail-touching component ? in-account,
  auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the
  binding [auditability invariant](#auditability-invariant-binding). | Verifiable privacy is the
  product's moat and shrinks future verification scope; it concentrates the trust-critical surface
  in one open, in-account place. |
209
210  ## Open items for the next iteration
211
212  - **Muneem handoff contract:** schema + transport (e.g. `UrlFetch` to a signed
213    endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.
214  - **Per-vertical "non-PII" definition** for receipts and bank statements (what is
215    sent vs. tokenized vs. kept in-account).
216  - **First extractor slice** to build in Phase 1 (recommend a narrow
217    "receipt-smelling" detector) and its golden-email test fixtures.
218

```

22. user (2026-06-10T18:00:29.191Z)

```

1  # Roadmap
2
3  Next steps for the modernised project, grouped by effort and grounded in the
4  current Apps Script / Gmail platform constraints.
5

```

```

6      ## Platform constraints that shape the design
7
8      For consumer (gmail.com) accounts:
9
10     - **6 minutes** max per execution; **90 minutes/day** total trigger runtime;
11       **20 triggers** per script per user; ~**20,000** Gmail read calls/day.
12       ([Apps Script quotas](https://developers.google.com/apps-script/guides/services/quotas))
13     - There is **no clearly documented daily Drive file-create quota**, so
14       `MAX_FILES_ADD_PER_DAY` (240) is a self-imposed heuristic, not an API limit.
15     - Gmail's
16       [`users.history.list`](https://developers.google.com/workspace/gmail/api/guides/sync)
17       with a stored `startHistoryId` is the purpose-built incremental sync (~2 quota
18       units vs 5 for a list call) and returns only changes. A `historyId` can expire
19       (HTTP 404 ? fall back to full sync), so it complements the date-window
20       backfill rather than replacing it.
21     - clasp can deploy from CI using a `CLASPRC_JSON` secret (contents of
22       `~/clasprc.json`); service accounts do **not** work with the Apps Script API,
23       so use a dedicated owner account's token.
24       ([clasp #225](https://github.com/google/clasp/issues/225))
25
26     ## Quick wins
27
28     - [x] **Cover the orchestration shells with tests.** `sync.ts` / `ui.ts` had no
29       coverage ? the trickiest logic (trigger fan-out, quota reset, catch-up loop,
30       `processInput` state reset). *(done in this branch; see `tests/sync.test.ts`,
31       `tests/ui.test.ts`)*
32     - [x] **Raise the per-run wall-clock budget.** Was capped at 1 minute against a
33       6-minute ceiling, making backfill ~6x slower than necessary. Now
34       `MAX_RUNTIME_MS` (5 min) in `src/config.ts`, threaded through `hasTime`.
35     - [x] **Guard against trigger leakage.** A `LockService` script lock prevents
36       concurrent runs from stacking triggers, deletion is scoped to the sync
37       handler, and a cap prunes any excess. *(done in this branch; `src/sync.ts`)*
38     - [x] **Add a Jest `coverageThreshold`** so CI fails if coverage regresses.
39       *(done; `jest.config.js`, gated at 85% stmts/lines/funcs, 75% branches,
40       `entry.ts` bundler glue excluded)*
41
42     ## Medium
43
44     - [x] **CI/CD auto-deploy on merge to `master`.** A gated `deploy` job writes
45       `CLASPRC_JSON` ? `~/clasprc.json` and runs `clasp push` / `clasp deploy`
46       against the committed `clasprc.json`. *(done; `.github/workflows/ci.yml`.
47       Requires only the `CLASPRC_JSON` repo secret; skips cleanly if unset.)*
48     - [x] **Cross-run dedup by message/attachment ID.** A bounded FIFO of processed
49       Gmail message ids (`src/processed.ts`, capped at `MAX_PROCESSED_IDS`) is
50       persisted in user properties; `updateDrive` skips messages already in the set
51       and reports newly-completed ids back to `sync.ts`. A message is only marked
52       done when all its attachments were copied cleanly (not truncated by the daily
53       cap or a transient error), so re-scanned windows skip the Drive
54       `getFilesByName` probe. *(done; `src/processed.ts`, `src/drive.ts`,
55       `src/sync.ts`, `src/state.ts`)*
56     - [x] **Observability.** Level-prefixed structured logging (`src/logger.ts`),
57       per-day run stats accumulator (`src/stats.ts` ? runs/files/errors/last error,
58       persisted via `PROP_RUN_STATS`), and an opt-in daily summary
59       (`src/notify.ts`) logged on each new-day rollover. The summary email is OFF by
60       default; enabling it needs `ENABLE_SUMMARY_EMAIL` + the `script.send_mail`
61       scope. *(done)*
62
63     ## Larger / architectural
64
65     - [x] **Hybrid incremental sync.** Keep the date-window scan for the initial
66       backfill; once caught up (`isToday`), switch the steady state to the Gmail
67       advanced service + `history.list`/`startHistoryId`, storing `historyId` in
68       properties and falling back to full sync on 404. *(done; pure logic in
69       `src/historySync.ts`, advanced-service shell in `src/gmailHistory.ts`, wired

```

```

69     into `src/sync.ts`. Gated behind `ENABLE_INCREMENTAL_SYNC` (OFF by default)
70     for gradual rollout over the proven backfill. The Gmail advanced service is
71     declared in `appsscript.json`; no new OAuth scope ? `gmail.readonly` covers
72     history reads. Backfill captures the mailbox `historyId` before processing so
73     mail arriving mid-run is still caught; expiry (HTTP 404) clears the cursor and
74     rewinds to a bounded date-window re-scan.)*
75     - [ ] **More structure options & Shared Drive support** ? by-sender, nested
76       `Year/Month`, label-driven folders; currently My Drive only.
77
78     ## Open-source & auditability readiness
79
80     Concrete prep so the [auditability invariant](docs/ARCHITECTURE.md#auditability-
invariant-binding)
81     is real, not aspirational, before/when the repo is opened up.
82
83     - [x] Root `LICENSE` file (MIT, matching the source headers).
84     - [x] Egress guard ? a test that fails CI if `src/` makes any network call
85       (`tests/auditability.test.ts`). Update it deliberately when the user-endpoint
86       egress lands (assert the allowlisted destination, don't remove the guard).
87     - [ ] `SECURITY.md` ? how to report issues + the current data-flow guarantees
88       (today: reads Gmail, writes the user's own Drive, **no egress**).
89     - [ ] ***"Deploy your own from source"*** guide ? self-host the GAS project end to
90       end, so the trust claim is "run the code you read."
91     - [ ] **Reproducible / verifiable build** ? pin the toolchain and document how to
92       diff a deployed `dist/Code.js` against a build of a given commit.
93     - [ ] `CONTRIBUTING.md` carrying the auditability PR checklist.
94

```

23. user (2026-06-10T18:00:34.661Z)

```

1     /*
2     * Bundle entry point. esbuild compiles this (with all imports) into a single
3     * IIFE assigned to the `GmailToDrive` global. A footer in esbuild.config.mjs
4     * then declares top-level `doGet` / `processInput` / `syncUserDrives` functions
5     * that delegate here, so the Apps Script runtime can resolve them by name for
6     * the web app, `google.script.run`, and time-based triggers.
7     */
8     export { doGet, processInput } from "./ui";
9     export { syncUserDrives } from "./sync";
10

```

24. user (2026-06-10T18:00:35.150Z)

```

1     import {
2         MAX_FILES_ADD_PER_DAY,
3         ONE_MINUTE,
4         ONE_YEAR,
5         PRIVACY_POLICY_URL,
6         SUPPORT_URL,
7     } from "./config";
8     import { logError } from "./logger";
9     import { isFolderStructure } from "./naming";
10    import * as state from "./state";
11    import { deleteAllTriggers, newSyncTrigger } from "./sync";
12
13    /** Bounds on the "how many years to sync" choice offered by the form. */
14    const MIN_SYNC_YEARS = 1;
15    const MAX_SYNC_YEARS = 10;
16
17    /** Web-app entry point: serves the setup form. */
18    export function doGet(): GoogleAppsScript.HTML.HtmlOutput {
19        return HtmlService.createHtmlOutputFromFile("index");
20    }
21
22    /** Shape of the form submitted from index.html. */

```

```

23 export interface SetupForm {
24     folderStructure: string;
25     syncFrom: string;
26 }
27
28 /**
29  * Called from the setup form. Resets this user's state (their Drive files are
30  * untouched), stores their preferences, and kicks off the first background
31  * sync. The web app runs as the accessing user (executeAs USER_ACCESSING), so
32  * everything here operates on the current user's own Gmail/Drive/properties.
33  *
34  * Wrapped so a permission problem (e.g. the user declined a scope on the
35  * granular consent screen) returns a clear message instead of a raw stack
36  * trace. Without all scopes granted the background trigger would otherwise fail
37  * silently later.
38  */
39 export function processInput(form: SetupForm): string {
40     // Validate the untrusted form before touching any state. An unknown structure
41     // would otherwise dump every file into the Drive root (with a logged error per
42     // attachment), and a non-numeric "years" would corrupt lastSynced.
43     const structure = form?.folderStructure ?? "";
44     const syncYears = Number.parseInt(form?.syncFrom ?? "", 10);
45     if (!isFolderStructure(structure)) {
46         return buildValidationErrorMessage("Please choose a folder structure.");
47     }
48     if (
49         !Number.isInteger(syncYears) ||
50         syncYears < MIN_SYNC_YEARS ||
51         syncYears > MAX_SYNC_YEARS
52     ) {
53         return buildValidationErrorMessage(
54             "Please choose how many years of attachments to sync.",
55         );
56     }
57
58     try {
59         // The user is (re)running setup; flush their past state and triggers.
60         state.clearAll();
61         deleteAllTriggers();
62
63         state.setFolderStructure(structure);
64
65         const startMillis = Date.now() - syncYears * ONE_YEAR;
66         state.setLastSynced(new Date(startMillis).toJSON());
67         state.setDebug("1");
68         state.setFilesAddedToday({ added: 0, date: new Date().toJSON() });
69
70         // One-time trigger so we return to the user immediately and sync in the
71         // background.
72         newSyncTrigger(2 * ONE_MINUTE);
73
74         return buildSuccessMessage();
75     } catch (e) {
76         logError("Setup failed: " + e);
77         return buildAuthErrorMessage();
78     }
79 }
80
81 function buildValidationErrorMessage(detail: string): string {
82     return (
83         "<br/><br/><br/><b>" +
84         detail +
85         "</b><br/><br/>Please go back, complete the form, and submit again."
86     );
87 }

```

```

88
89 function buildSuccessMessage(): string {
90     return (
91         "<br/><br/><br/>You're all set ? Gmail Attachments to Drive will start " +
92         "copying your attachments into a <b>GmailAttachments</b> folder in " +
93         "<i>your</i> Google Drive within a minute." +
94         "<br/><br/>Google Apps Script limits us to " +
95         MAX_FILES_ADD_PER_DAY +
96         " files per day and caps daily processing time, so the first sync may take " +
97         "a couple of days to catch up. After that, new attachments usually appear " +
98         "within 5?10 minutes." +
99         "<br/><br/>Tip: install Google Drive for desktop and link this account, so " +
100        "these attachments sync to your computer automatically." +
101        "<br/><br/><b>Privacy:</b> the app reads your Gmail attachments and copies " +
102        "them into your own Drive. Nothing leaves your Google account. " +
103        '<a href="' +
104        PRIVACY_POLICY_URL +
105        '" target="_blank">Privacy policy</a>.' +
106        "<br/><br/>Questions or issues? " +
107        '<a href="' +
108        SUPPORT_URL +
109        '" target="_blank">Project page & support</a>.'
110    );
111 }
112
113 function buildAuthErrorMessage(): string {
114     return (
115         "<br/><br/><br/><b>Setup couldn't finish.</b>" +
116         "<br/><br/>This usually means not all permissions were granted. Please " +
117         "reload this page and, when Google asks, <b>allow all</b> the requested " +
118         "permissions (read your Gmail, manage your Drive files, and run in the " +
119         "background) so the app can sync your attachments." +
120         "<br/><br/>If it keeps failing, see the " +
121         '<a href="' +
122         SUPPORT_URL +
123         '" target="_blank">project page</a>.'
124     );
125 }
126

```

25. user (2026-06-10T18:00:35.375Z)

```

1    /** Bookkeeping for how many files we have added on a given calendar day. */
2    export interface FilesAddedToday {
3        added: number;
4        /** ISO timestamp of the day this count belongs to. */
5        date: string;
6    }
7
8    /**
9     * True while we are still within our wall-clock budget for a single execution
10     * (Apps Script kills long-running scripts). `now` and `budgetMs` are injected
11     * for testability.
12     */
13    export function hasTime(now: Date, start: Date, budgetMs: number): boolean {
14        return now.getTime() - start.getTime() < budgetMs;
15    }
16
17    /**
18     * Whether two dates fall on the same calendar day. Unlike the legacy code
19     * (which compared only day-of-month), this also compares month and year, so
20     * the daily counter resets correctly across month boundaries.
21     */
22    export function isSameCalendarDay(a: Date, b: Date): boolean {
23        return (

```

```

24         a.getFullYear() === b.getFullYear() &&
25         a.getMonth() === b.getMonth() &&
26         a.getDate() === b.getDate()
27     );
28 }
29

```

26. user (2026-06-10T18:00:39.845Z)

```

1     import {
2         PROP_RUN_STATS,
3         PROP_SUMMARY_LAST_SENT,
4     } from "../src/config";
5     import { maybeSendDailySummary } from "../src/notify";
6
7     type Store = Record<string, string>;
8
9     function fakeProps(initial: Store) {
10         const store: Store = { ...initial };
11         return {
12             store,
13             getProperty: (k: string) => (k in store ? store[k] : null),
14             setProperty: (k: string, v: string) => {
15                 store[k] = String(v);
16             },
17             deleteAllProperties: () => {
18                 for (const k of Object.keys(store)) delete store[k];
19             },
20         };
21     }
22
23     const g = globalThis as unknown as Record<string, unknown>;
24
25     function install(initial: Store) {
26         const props = fakeProps(initial);
27         g.PropertiesService = { getUserProperties: () => props };
28         return props;
29     }
30
31     const priorDayStats = JSON.stringify({
32         day: "2012-06-05",
33         runs: 3,
34         filesAdded: 7,
35         errors: 0,
36         lastError: "",
37     });
38
39     beforeEach(() => {
40         (g.MailApp as { sendEmail: jest.Mock }).sendEmail.mockClear();
41     });
42
43     describe("maybeSendDailySummary", () => {
44         const now = new Date(2012, 5, 6, 9, 0, 0); // 2012-06-06
45
46         it("is a no-op when already summarised today", () => {
47             install({
48                 [PROP_SUMMARY_LAST_SENT]: "2012-06-06",
49                 [PROP_RUN_STATS]: priorDayStats,
50             });
51
52             maybeSendDailySummary(now);
53
54             expect((g.MailApp as { sendEmail: jest.Mock }).sendEmail).not.toHaveBeenCalled();
55         });
56

```

```

57     it("summarises the prior day and marks today sent", () => {
58         const props = install({ [PROP_RUN_STATS]: priorDayStats });
59
60         maybeSendDailySummary(now);
61
62         // Email stays off by default (ENABLE_SUMMARY_EMAIL=false), but the marker
63         // advances so we don't summarise the same rollover twice.
64         expect(props.store[PROP_SUMMARY_LAST_SENT]).toBe("2012-06-06");
65         expect((g.MailApp as { sendEmail: jest.Mock }).sendEmail).not.toHaveBeenCalled();
66     });
67
68     it("does not summarise when the only stats are for today", () => {
69         const todayStats = JSON.stringify({
70             day: "2012-06-06",
71             runs: 1,
72             filesAdded: 0,
73             errors: 0,
74             lastError: "",
75         });
76         const props = install({ [PROP_RUN_STATS]: todayStats });
77
78         maybeSendDailySummary(now);
79
80         // Nothing from a prior day; still arms the marker for the next rollover.
81         expect(props.store[PROP_SUMMARY_LAST_SENT]).toBe("2012-06-06");
82     });
83 });
84

```

27. user (2026-06-10T18:00:40.905Z)

```

1  {
2      "name": "gmail-attachments-to-gdrive",
3      "version": "2.0.0",
4      "description": "Google Apps Script that syncs Gmail attachments into Google Drive.",
5      "license": "MIT",
6      "private": true,
7      "scripts": {
8          "build": "node esbuild.config.mjs",
9          "typecheck": "tsc --noEmit",
10         "lint": "eslint src tests",
11         "test": "jest",
12         "test:coverage": "jest --coverage",
13         "push": "npm run build && clasp push",
14         "deploy": "npm run build && clasp deploy"
15     },
16     "devDependencies": {
17         "@google/clasp": "^2.4.2",
18         "@types/google-apps-script": "^1.0.83",
19         "@types/jest": "^29.5.12",
20         "@types/node": "^20.12.7",
21         "esbuild": "^0.21.5",
22         "eslint": "^9.5.0",
23         "jest": "^29.7.0",
24         "ts-jest": "^29.1.4",
25         "typescript": "^5.4.5",
26         "typescript-eslint": "^8.0.0"
27     }
28 }
29

```

28. user (2026-06-10T18:00:41.250Z)

```

1  /** @type {import('ts-jest').JestConfigWithTsJest} */
2  module.exports = {

```

```

3     preset: "ts-jest",
4     testEnvironment: "node",
5     setupFiles: [<rootDir>/tests/setup.ts"],
6     testMatch: ["**/tests/**/*.test.ts"],
7     collectCoverageFrom: [
8       "src/**/*.ts",
9       // entry.ts is bundler glue (re-exports only); it has no logic to test.
10      "!src/entry.ts",
11      // Ambient type declarations carry no runtime code.
12      "!src/**/*.d.ts",
13    ],
14    coverageThreshold: {
15      global: {
16        statements: 85,
17        branches: 75,
18        functions: 85,
19        lines: 85,
20      },
21    },
22  };
23

```

29. user (2026-06-10T18:00:41.730Z)

```

1  /*
2   * Minimal ambient declarations for the parts of the Gmail advanced service we
3   * use. `@types/google-apps-script` types the built-in `GmailApp` but not the
4   * advanced `Gmail` service, which the runtime injects as a global when enabled
5   * in appsscript.json. We declare only what `gmailHistory.ts` calls.
6   */
7  declare namespace GmailAdvanced {
8    interface Profile {
9      emailAddress?: string;
10     historyId?: string;
11   }
12
13   interface HistoryMessageRef {
14     id?: string;
15     threadId?: string;
16   }
17
18   interface HistoryItem {
19     id?: string;
20     messagesAdded?: { message?: HistoryMessageRef }[];
21   }
22
23   interface HistoryList {
24     history?: HistoryItem[];
25     historyId?: string;
26     nextPageToken?: string;
27   }
28
29   interface HistoryListOptions {
30     startHistoryId: string;
31     historyTypes?: string[];
32     maxResults?: number;
33     pageToken?: string;
34   }
35
36   interface UsersCollection {
37     getProfile(userId: string): Profile;
38     History: {
39       list(userId: string, optionalArgs?: HistoryListOptions): HistoryList;
40     };
41   }

```

```

42
43     interface GmailService {
44         Users: UsersCollection;
45     }
46 }
47
48 declare const Gmail: GmailAdvanced.GmailService;
49

```

30. user (2026-06-10T18:00:47.778Z)

```

1      /*
2      * Installs no-op / jest.fn() stand-ins for the Apps Script globals so modules
3      * that reference them can be imported in a plain Node test environment. Pure
4      * logic modules (query, naming, contentTypes, quota) do not need these, but the
5      * shells (drive, sync, ui, state) do.
6      */
7
8      const g = globalThis as unknown as Record<string, unknown>;
9
10     g.Logger = { log: jest.fn() };
11
12     g.Utilities = { sleep: jest.fn() };
13
14     // Individual tests override these with richer fakes as needed.
15     g.PropertiesService = {
16         getUserProperties: jest.fn(),
17         getScriptProperties: jest.fn(),
18     };
19     g.DriveApp = {
20         getFolders: jest.fn(),
21         createFolder: jest.fn(),
22     };
23     g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
24     g.Gmail = {
25         Users: {
26             getProfile: jest.fn(() => ({ historyId: "1" })),
27             History: { list: jest.fn(() => ({ history: [], historyId: "1" })) },
28         },
29     };
30     g.ScriptApp = {
31         newTrigger: jest.fn(() => ({
32             timeBased: () => ({ after: () => ({ create: jest.fn() }) }),
33         })),
34         getProjectTriggers: jest.fn(() => []),
35         deleteTrigger: jest.fn(),
36     };
37     g.HtmlService = { createHtmlOutputFromFile: jest.fn() };
38     g.LockService = {
39         getUserLock: jest.fn(() => ({
40             tryLock: jest.fn(() => true),
41             releaseLock: jest.fn(),
42         })),
43     };
44     g.MailApp = { sendEmail: jest.fn() };
45     g.Session = {
46         getEffectiveUser: jest.fn(() => ({ getEmail: () => "me@example.com" })),
47     };
48

```

31. user (2026-06-10T18:00:48.265Z)

```

1      import { readFileSync, readdirSync } from "fs";
2      import { join } from "path";
3

```

```

4      /*
5       * Enforces the auditability invariant (docs/ARCHITECTURE.md): this repo is the
6       * only code that touches a user's Gmail, and it makes NO network egress today.
7       *
8       * The ONLY sanctioned way data may ever leave a user's account is a deliberate,
9       * reviewed REDACT/MINIMIZE -> user's-own-endpoint path. When that lands, update
10      * this guard to assert the single allowlisted destination ? do not delete it.
11      */
12
13      // Apps Script's only outbound-network primitive.
14      const FORBIDDEN = /UrlFetchApp/;
15
16      function tsFiles(dir: string): string[] {
17          return readdirSync(dir, { withFileTypes: true }).flatMap((entry) => {
18              const full = join(dir, entry.name);
19              if (entry.isDirectory()) return tsFiles(full);
20              return entry.name.endsWith(".ts") ? [full] : [];
21          });
22      }
23
24      describe("auditability invariant", () => {
25          it("src/ makes no outbound network calls (no UrlFetchApp)", () => {
26              const offenders = tsFiles("src").filter((file) =>
27                  FORBIDDEN.test(readFileSync(file, "utf8")),
28              );
29              expect(offenders).toEqual([]);
30          });
31      });
32

```

32. user (2026-06-10T18:00:48.637Z)

```

1      import { build } from "esbuild";
2      import { copyFileSync, mkdirSync } from "node:fs";
3
4      mkdirSync("dist", { recursive: true });
5
6      await build({
7          entryPoints: ["src/entry.ts"],
8          bundle: true,
9          format: "iife",
10         // Expose the module's exports as a single global the footer delegates into.
11         globalName: "GmailToDrive",
12         // Apps Script runs on V8; es2019 is a safe floor and lets esbuild downlevel
13         // newer syntax (e.g. ??). Whitespace may be trimmed, but identifiers MUST be
14         // preserved so trigger/web-app name lookups keep working.
15         target: "es2019",
16         minifyIdentifiers: false,
17         outfile: "dist/Code.js",
18         // Real top-level function declarations: they resolve by name at runtime AND
19         // show up in the Apps Script editor's Run dropdown.
20         footer: {
21             js: [
22                 "function doGet(e) { return GmailToDrive.doGet(e); }",
23                 "function processInput(form) { return GmailToDrive.processInput(form); }",
24                 "function syncUserDrives(e) { return GmailToDrive.syncUserDrives(e); }",
25             ].join("\n"),
26         },
27     });
28
29     // clasp pushes everything under dist/, so the manifest and HTML live there too.
30     copyFileSync("appsscript.json", "dist/appsscript.json");
31     copyFileSync("html/index.html", "dist/index.html");
32
33     console.log("Build complete -> dist/Code.js");

```

33. assistant (2026-06-10T18:06:56.930Z)

I've read all the relevant sources. Now let me check the toolchain baseline and the git remote.

34. user (2026-06-10T18:06:59.250Z)

False

origin <https://github.com/avidullu/gmail-attachments-to-gdrive.git> (fetch)

origin <https://github.com/avidullu/gmail-attachments-to-gdrive.git> (push)

35. user (2026-06-10T18:07:20.688Z)

npm notice

npm notice New minor version of npm available! 11.13.0 -> 11.16.0

npm notice Changelog: <https://github.com/npm/cli/releases/tag/v11.16.0>

npm notice To update run: npm install -g npm@11.16.0

npm notice

36. user (2026-06-10T18:07:40.079Z)

PASS tests/stats.test.ts (6.445 s)

PASS tests/query.test.ts (6.529 s)

PASS tests/contentTypes.test.ts (6.58 s)

PASS tests/auditability.test.ts (6.551 s)

PASS tests/drive.test.ts (6.604 s)

PASS tests/notify.test.ts (6.577 s)

PASS tests/ui.test.ts (6.664 s)

PASS tests/sync.test.ts (6.677 s)

PASS tests/syncIncremental.test.ts (6.693 s)

Test Suites: 14 passed, 14 total

Tests: 89 passed, 89 total

Snapshots: 0 total

Time: 8.107 s

Ran all test suites.

37. assistant (2026-06-10T18:07:45.622Z)

Baseline is green: 14 suites, 89 tests. Now I'll create the first branch and implement F1+F2.

38. user (2026-06-10T18:07:47.746Z)

Switched to a new branch 'fix/sync-data-loss'

39. assistant (2026-06-10T18:07:54.010Z)

First, the config and state plumbing for the consecutive-dirty-pass counter.

40. user (2026-06-10T18:07:56.681Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\config.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

41. user (2026-06-10T18:08:04.085Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

42. user (2026-06-10T18:08:11.782Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts has been updated

successfully. (file state is current in your context ? no need to Read it back)

43. assistant (2026-06-10T18:08:16.278Z)

Now the core sync.ts changes (F1 + F2a).

44. user (2026-06-10T18:08:17.722Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

45. user (2026-06-10T18:08:50.780Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

46. user (2026-06-10T18:08:56.934Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

47. user (2026-06-10T18:09:04.872Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

48. user (2026-06-10T18:09:11.251Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

49. user (2026-06-10T18:09:22.800Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

50. user (2026-06-10T18:09:28.184Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\types\gmailAdvanced.d.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

51. assistant (2026-06-10T18:09:33.652Z)

Now the tests. First the pure layer (historySync, gmailHistory).

52. user (2026-06-10T18:09:35.855Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

53. user (2026-06-10T18:09:46.041Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

54. user (2026-06-10T18:09:54.464Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-06-10T18:10:03.428Z)

Now the sync shell tests for the backfill cursor hold.

56. user (2026-06-10T18:10:06.511Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

57. user (2026-06-10T18:10:20.395Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

58. assistant (2026-06-10T18:10:32.446Z)

Now the incremental-mode shell tests.

59. user (2026-06-10T18:10:35.596Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

60. user (2026-06-10T18:11:04.334Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

61. user (2026-06-10T18:11:17.810Z)

PASS tests/contentTypes.test.ts
PASS tests/stats.test.ts
PASS tests/naming.test.ts
PASS tests/drive.test.ts
PASS tests/processed.test.ts
PASS tests/auditability.test.ts
PASS tests/quota.test.ts
PASS tests/gmailHistory.test.ts
PASS tests/query.test.ts
PASS tests/historySync.test.ts
PASS tests/notify.test.ts
PASS tests/sync.test.ts (5.008 s)
PASS tests/ui.test.ts
PASS tests/syncIncremental.test.ts (5.043 s)

Test Suites: 14 passed, 14 total
Tests: 99 passed, 99 total
Snapshots: 0 total
Time: 6.314 s, estimated 7 s
Ran all test suites.

62. user (2026-06-10T18:11:34.434Z)

PASS tests/gmailHistory.test.ts (6.405 s)
PASS tests/syncIncremental.test.ts (6.492 s)
PASS tests/sync.test.ts (6.526 s)
PASS tests/ui.test.ts (6.599 s)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	93.99	88.72	98.63	94.29	
config.ts	100	100	100	100	
contentTypes.ts	100	100	100	100	
drive.ts	85.88	66.66	100	87.8	36-39, 60, 93-94, 142-143, 149
gmailHistory.ts	100	100	100	100	
historySync.ts	100	100	100	100	
logger.ts	88.88	0	100	88.88	15
naming.ts	100	100	100	100	

notify.ts		76.19		50		100		76.19		34-40
processed.ts		100		100		100		100		
query.ts		100		100		100		100		
quota.ts		100		100		100		100		
state.ts		96.22		66.66		95.65		96.22		53,128
stats.ts		100		85.18		100		100		44-47
sync.ts		91.79		92.68		100		91.72		42,56,98-99,114-115,204,265-268
ui.ts		100		100		100		100		
-----		-----		-----		-----		-----		-----

Test Suites: 14 passed, 14 total
 Tests: 99 passed, 99 total
 Snapshots: 0 total
 Time: 8.314 s
 Ran all test suites.

63. user (2026-06-10T18:11:49.783Z)

```
> gmail-attachments-to-gdrive@2.0.0 typecheck
> tsc --noEmit
```

```
> gmail-attachments-to-gdrive@2.0.0 lint
> eslint src tests
```

```
> gmail-attachments-to-gdrive@2.0.0 build
> node esbuild.config.mjs
```

Build complete -> dist/Code.js

64. assistant (2026-06-10T18:11:55.595Z)

All green. Commit and create the first PR.

65. user (2026-06-10T18:11:58.532Z)

```
M src/config.ts
M src/gmailHistory.ts
M src/historySync.ts
M src/state.ts
M src/sync.ts
M src/types/gmailAdvanced.d.ts
M tests/gmailHistory.test.ts
M tests/historySync.test.ts
M tests/sync.test.ts
M tests/syncIncremental.test.ts
```

66. user (2026-06-10T18:12:13.224Z)

```
[fix/sync-data-loss 1b32115] fix(sync): never advance a sync cursor past unfinished work
10 files changed, 455 insertions(+), 17 deletions(-)
remote:
remote: Create a pull request for 'fix/sync-data-loss' on GitHub by visiting:
remote:   https://github.com/avidullu/gmail-attachments-to-gdrive/pull/new/fix/sync-data-loss
remote:
remote: GitHub found 3 vulnerabilities on avidullu/gmail-attachments-to-gdrive's default branch
(1 high, 2 moderate). To find out more, visit:
remote:   https://github.com/avidullu/gmail-attachments-to-gdrive/security/dependabot
remote:
branch 'fix/sync-data-loss' set up to track 'origin/fix/sync-data-loss'.
To https://github.com/avidullu/gmail-attachments-to-gdrive.git
* [new branch]      fix/sync-data-loss -> fix/sync-data-loss
```

67. user (2026-06-10T18:12:41.003Z)

<https://github.com/avidullu/gmail-attachments-to-gdrive/pull/15>

68. assistant (2026-06-10T18:12:48.436Z)

PR1 is up: <https://github.com/avidullu/gmail-attachments-to-gdrive/pull/15>. Now the second branch off master for F3.

69. user (2026-06-10T18:12:50.398Z)

Your branch is up to date with 'origin/master'.
Switched to branch 'master'
Switched to a new branch 'fix/operational-defaults'

70. user (2026-06-10T18:13:02.155Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

71. user (2026-06-10T18:13:07.734Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

72. assistant (2026-06-10T18:13:12.833Z)

F3(b) ? timezone, and F3(c) ? the recipient resolution in notify.ts.

73. user (2026-06-10T18:13:14.313Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\appsscript.json has been updated successfully. (file state is current in your context ? no need to Read it back)

74. user (2026-06-10T18:13:22.917Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

75. user (2026-06-10T18:13:28.450Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

76. assistant (2026-06-10T18:13:35.864Z)

Now the test updates.

77. user (2026-06-10T18:13:37.763Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\setup.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

78. user (2026-06-10T18:13:45.920Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

79. user (2026-06-10T18:13:56.443Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

80. user (2026-06-10T18:14:07.377Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

81. user (2026-06-10T18:14:14.000Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

82. user (2026-06-10T18:14:20.619Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

83. user (2026-06-10T18:14:29.344Z)

The file C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts has been updated successfully. (file state is current in your context ? no need to Read it back)

84. assistant (2026-06-10T18:14:44.050Z)

```
// Enable the opt-in summary email for this suite only (the same pattern as
// tests/syncIncremental.test.ts uses for ENABLE_INCREMENTAL_SYNC).
jest.mock("../src/config", () => ({
  ...jest.requireActual("../src/config"),
  ENABLE_SUMMARY_EMAIL: true,
}));

import { PROP_RUN_STATS } from "../src/config";
import { maybeSendDailySummary } from "../src/notify";

type Store = Record<string, string>;

function fakeProps(initial: Store) {
  const store: Store = { ...initial };
  return {
    store,
    getProperty: (k: string) => (k in store ? store[k] : null),
    setProperty: (k: string, v: string) => {
      store[k] = String(v);
    },
  };
}

const g = globalThis as unknown as Record<string, unknown>;

const priorDayStats = JSON.stringify({
  day: "2012-06-05",
  runs: 3,
  filesAdded: 7,
  errors: 0,
  lastError: "",
});

describe("maybeSendDailySummary ? email enabled", () => {
  it("mails the summary to the executing user's own address", () => {
    const props = fakeProps({ [PROP_RUN_STATS]: priorDayStats });
    g.PropertiesService = { getUserProperties: () => props };
    const sendEmail = jest.fn();
    g.MailApp = { sendEmail };
    g.Session = {
      getEffectiveUser: () => ({ getEmail: () => "me@example.com" }),
    };
  });
});
```

```

maybeSendDailySummary(new Date(2012, 5, 6, 9, 0, 0));

// SUMMARY_EMAIL_RECIPIENT is empty by default, so the recipient is the
// executing user ? never a shared address in a multi-user deployment.
expect(sendEmail).toHaveBeenCalledTimes(1);
expect(sendEmail.mock.calls[0][0]).toBe("me@example.com");
expect(sendEmail.mock.calls[0][1]).toContain("2012-06-05");
});
});

```

85. user (2026-06-10T18:14:44.074Z)

File created successfully at: C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notifyEmail.test.ts (file state is current in your context ? no need to Read it back)

86. user (2026-06-10T18:15:05.215Z)

PASS tests/notify.test.ts (6.52 s)
 PASS tests/syncIncremental.test.ts (6.596 s)
 PASS tests/sync.test.ts (6.644 s)
 PASS tests/ui.test.ts (6.689 s)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	93.98	91.16	98.57	94.31	
config.ts	100	100	100	100	
contentTypes.ts	100	100	100	100	
drive.ts	83.52	72.22	100	85.36	62, 98-99, 147-148, 154, 171-176
gmailHistory.ts	100	100	100	100	
historySync.ts	100	100	100	100	
logger.ts	88.88	0	100	88.88	15
naming.ts	100	100	100	100	
notify.ts	95.65	100	100	95.65	57
processed.ts	100	100	100	100	
query.ts	100	100	100	100	
quota.ts	100	100	100	100	
state.ts	95.74	66.66	95.23	95.74	52, 112
stats.ts	100	85.18	100	100	44-47
sync.ts	90.26	90.9	100	90.17	42, 56, 98-99, 114-115, 149, 227-230
ui.ts	100	100	100	100	

Test Suites: 15 passed, 15 total
 Tests: 94 passed, 94 total
 Snapshots: 0 total
 Time: 8.575 s
 Ran all test suites.

87. user (2026-06-10T18:15:16.382Z)

```

> gmail-attachments-to-gdrive@2.0.0 typecheck
> tsc --noEmit

```

```

> gmail-attachments-to-gdrive@2.0.0 lint
> eslint src tests

```

```

> gmail-attachments-to-gdrive@2.0.0 build
> node esbuild.config.mjs

```

Build complete -> dist/Code.js

88. user (2026-06-10T18:15:22.956Z)

```
warning: in the working copy of 'tests/notifyEmail.test.ts', LF will be replaced by CRLF the
next time Git touches it
M  appsscript.json
M  src/drive.ts
M  src/notify.ts
M  tests/drive.test.ts
M  tests/notify.test.ts
A  tests/notifyEmail.test.ts
M  tests/setup.ts
M  tests/sync.test.ts
M  tests/syncIncremental.test.ts
```

89. user (2026-06-10T18:15:37.101Z)

```
[fix/operational-defaults 39c13b5] fix(ops): scope Drive folder probe to root, Asia/Kolkata
timezone, lock summary recipient
 9 files changed, 134 insertions(+), 10 deletions(-)
 create mode 100644 tests/notifyEmail.test.ts
remote:
remote: Create a pull request for 'fix/operational-defaults' on GitHub by visiting:
remote:      https://github.com/avidullu/gmail-attachments-to-gdrive/pull/new/fix/operational-
defaults
remote:
remote: GitHub found 3 vulnerabilities on avidullu/gmail-attachments-to-gdrive's default branch
(1 high, 2 moderate). To find out more, visit:
remote:      https://github.com/avidullu/gmail-attachments-to-gdrive/security/dependabot
remote:
branch 'fix/operational-defaults' set up to track 'origin/fix/operational-defaults'.
To https://github.com/avidullu/gmail-attachments-to-gdrive.git
 * [new branch]      fix/operational-defaults -> fix/operational-defaults
```

90. user (2026-06-10T18:16:00.045Z)

<https://github.com/avidullu/gmail-attachments-to-gdrive/pull/16>

91. assistant (2026-06-10T18:16:05.632Z)

Both PRs are up. Final housekeeping: return the repo to master and verify state.

92. user (2026-06-10T18:16:08.319Z)

```
Your branch is up to date with 'origin/master'.
Switched to branch 'master'
16      fix(ops): root-scoped Drive folder probe, Asia/Kolkata timezone, summary-recipient
default (F3) fix/operational-defaults      OPEN      2026-06-10T18:15:58Z
15      fix(sync): never advance a sync cursor past unfinished work (F1+F2)      fix/sync-data-
loss      OPEN      2026-06-10T18:12:40Z
```

93. assistant (2026-06-10T18:16:31.547Z)

Done. Both PRs are open, neither merged, master is clean.

PR URLs

```
- **PR #15 (F1+F2):** https://github.com/avidullu/gmail-attachments-to-gdrive/pull/15 ? branch
`fix/sync-data-loss`
- **PR #16 (F3):** https://github.com/avidullu/gmail-attachments-to-gdrive/pull/16 ? branch
`fix/operational-defaults` (independent, off master)
```

What changed per fix

```
**F1 ? backfill cursor (src/sync.ts):** a 30-day window with attachment-level errors no longer
```

advances `lastSynced`; the loop stops (retrying in-run would spin on the same failure inside the 6-min budget) and the next run re-scans the window, leaning on the processed-ids FIFO + Drive `getFilesByName` probe for cheap, duplicate-safe re-scans. Cap-truncation was already held by the existing `added < MAX` guard. A dirty final window also blocks the backfill?incremental `historyId` capture.

****F2a ? incremental cursor (src/sync.ts):**** `setHistoryId(changes.historyId)` now runs only when every reported message was handled cleanly (no errors, no cap truncation, including the latent skipped-entirely branch). Dirty runs hold the cursor; next run re-lists the same span. Per-record anchoring was evaluated (records do carry ids) but rejected ? attributing deleted/spam/already-processed skips back to records adds disproportionate machinery; the finding explicitly blesses hold-on-dirty.

****F2b ? (src/gmailHistory.ts):**** `GmailApp.getMessageById` wrapped in try/catch (it throws for deleted/inaccessible messages); one vanished message now skips with an INFO log instead of wedging every incremental run until cursor expiry. Combined with F2a, an all-deleted span still advances.

****F2c ? (src/historySync.ts):**** `collectAddedMessageIds` filters `SPAM`/`TRASH` via `labelIds` already on the history message stubs (zero extra quota), matching the backfill's `-in:trash -in:spam`.

****Wedge visibility:**** persisted consecutive-dirty-pass counter (`PROP\_DIRTY\_PASSES`) ? incremented + `logInfo`'d when a cursor is held on errors, reset on clean passes, untouched on crashed runs.

****F3a (src/drive.ts):**** head-folder probe now uses `DriveApp.getRootFolder()` instead of `DriveApp` (whose `getFolders()` iterates every folder in Drive, not root children). ****F3b (appsscript.json):**** timeZone `America/Los\_Angeles` ? `Asia/Kolkata`. ****F3c (src/notify.ts):**** recipient rule extracted into pure `resolveSummaryRecipient(override, userEmail)` + shell test of the enabled path.

Test counts

- Baseline (master): 14 suites, ****89 tests****
- PR #15 branch: 14 suites, ****99 tests**** (+10)
- PR #16 branch: 15 suites, ****94 tests**** (+5)
- Both branches: `npm test` green, coverage gate (85/75) satisfied, `typecheck`/`lint`/`build` pass. Test fakes in the two PRs were written so they merge in either order.

Findings adjusted

- ****F3c was partially stale:**** at HEAD, `SUMMARY\_EMAIL\_RECIPIENT` already defaulted to `""` with a `Session.getEffectiveUser()` fallback ? the multi-user mail-to-one-address bug did not exist as described. The PR adds the pure extraction + regression tests the finding asked for, and deliberately keeps `getEffectiveUser` over the suggested `getActiveUser` (which is typically empty in time-trigger context; the effective user is the trigger installer, i.e. the user themself). Stated in the PR body.
- ****F1 cap-truncation half was already safe:**** the existing `added < MAX` guard already held the cursor on cap-truncated windows; only the error case needed fixing. Noted in the PR body.